

**NATIONAL ECONOMICS UNIVERSITY
COLLEGE OF TECHNOLOGY**



**APPLYING MACHINE LEARNING TECHNIQUES FOR
CREDIT SCORE PREDICTION**

Name: Nguyen Thu Tra

Student ID: 11226306

Class: DSEB 64A

Ha Noi, 2025

Contents

A. INTRODUCTION.....	5
<i>I. Background</i>	5
<i>II. Objectives</i>	6
<i>III. Scope</i>	7
B. THEORETICAL BACKGROUND.....	9
<i>I. Theoretical Review</i>	9
<i>II. Methodology</i>	10
1. Data Preprocessing	10
1.1. Data Cleaning	10
1.2. Feature Selection	10
1.3. Encoding Categorical Variables	10
2. Feature Engineering with WOE and IV	10
2.1. Binning	10
2.2. WOE Calculation	12
2.3. IV Calculation	13
2.4. Integration with Machine Learning Models	15
2.5. Practical Considerations.....	16
3. Model Development	16
3.1. Data Preparation.....	17
3.2. Logistic Regression	17
3.3. Random Forest.....	18
3.4. GridSearchCV for Hyperparameter Tuning.....	19
3.5. Training and Validation	21
3.6. Practical Considerations.....	21
4. Model Evaluation	22
4.1. Evaluation Framework	22
4.2. Accuracy	23
4.3. Precision	23

4.4. Recall	24
4.5. F1-Score	24
4.6 AUC-ROC	25
4.7. Gini Coefficient	25
4.8. Comparative Analysis.....	26
C. DATA.....	27
I. Data Description	27
II. Data Preprocessing and Exploratory Data Analysis	28
1. Identification of Numeric and Categorical Columns	28
2. Conversion of Categorical Columns to Numeric	29
3. Additional Numeric Conversion with Error Handling	31
4. Visualize outliers for each numeric column	32
5. Correlation Heatmap.....	37
5.1. Strong Positive Correlations.....	37
5.2. Moderate Negative Correlations.....	38
5.3. Weak Positive Correlations	39
5.4. Weak Negative Correlations	39
5.5. Near-Zero Correlations	40
6. Analysis of Unique Values in Categorical Features	40
7. Distribution of Categorical Features via Bar Charts	42
8. Handling Missing Value.....	45
8.1. Unified Outlier and Missing Value Handling for Numeric Features	47
9. Handling Outliers.....	48
10. Feature Engineering and Selection Using Weight of Evidence (WOE) and Information Value (IV).....	49
10.1. Methodology for WOE and IV Calculation	50
10.2. Detailed Results and Feature-Specific Insights	52
10.3. Detailed Insights and Implications	53
10.4. Practical Considerations.....	56

D. MODEL.....	58
<i>I. Feature Engineering with WOE Encoding and Logistic Regression Model Development</i>	58
1. WOE Encoding for Categorical Features	58
2. Logistic Regression Model Development	59
3. Model Evaluation and Results	60
4. Detailed Insights and Implications	61
5. Practical Considerations	63
<i>II. Random Forest Model Development and Evaluation</i>	65
1. Random Forest Model Development.....	66
2. Model Evaluation and Results	67
3. Detailed Insights and Implications	68
4. Practical Considerations	70
<i>III. Conclusion and Discussion</i>	71
<i>IV. Discussion of Results.....</i>	72
1. Implications for Credit Risk Assessment	73
2. Limitations and Recommendations for Future Work	74
E. CODE.....	75

A. INTRODUCTION

I. Background

Credit scoring is a pivotal component of the financial industry, serving as a critical tool for assessing the creditworthiness of individuals seeking loans, credit cards, or other financial products. Financial institutions rely on credit scores to evaluate the likelihood of timely repayment, thereby mitigating risks associated with lending. Traditionally, credit scoring models have been built using statistical techniques, such as logistic regression, which analyze historical data to predict repayment behavior. However, the advent of machine learning has revolutionized this domain by introducing advanced algorithms capable of uncovering complex patterns in large datasets, enhancing predictive accuracy, and adapting to dynamic economic conditions.

The dataset under consideration contains a rich set of features, including demographic details (e.g., age, occupation), financial metrics (e.g., annual income, monthly in-hand salary, outstanding debt), and credit-related behaviors (e.g., credit utilization ratio, payment behavior, credit history age). These variables provide a comprehensive view of an individual's financial profile, making it an ideal candidate for predictive modeling. However, raw data often requires transformation to maximize its utility in machine learning models. Techniques such as Weight of Evidence (WOE) and Information Value (IV) are widely used in credit scoring to preprocess and quantify the predictive power of variables, enabling better feature selection and model interpretability.

WOE transforms categorical and continuous variables into a logarithmic scale that measures the relative risk of default across variable bins, while IV quantifies the strength of each variable's association with the target outcome (e.g., credit score categories: Poor, Standard, Good). These techniques are particularly valuable in handling imbalanced datasets and ensuring that models focus on the most predictive features. Machine learning algorithms, such as Logistic Regression and Random Forest, further complement WOE and IV by leveraging their outputs to build robust predictive models. Logistic Regression offers interpretability and is well-suited for binary or ordinal outcomes, while Random Forest excels in capturing non-linear relationships and interactions among variables, making it adept at handling complex datasets.

The motivation for this study stems from the growing need for accurate and efficient credit scoring systems in an era of expanding credit access and rising financial complexities. Traditional models often struggle with high-dimensional data and fail to account for nuanced patterns in borrower behavior. Machine learning, combined with WOE and IV, offers a promising solution by enhancing predictive performance, reducing default rates, and enabling fairer lending decisions. This project seeks to harness these techniques to develop a credit score prediction model that is both accurate and interpretable, contributing to the evolution of credit risk assessment in the financial sector.

II. Objectives

The primary goal of this study is to develop a robust and accurate credit score prediction model using machine learning techniques, with a focus on Logistic Regression and Random Forest, augmented by WOE and IV preprocessing. The specific objectives are as follows:

1. **To preprocess the dataset using WOE and IV techniques:** Transform the dataset's features (e.g., age, annual income, credit utilization ratio, payment behavior) into WOE values and calculate IV scores to identify the most predictive variables for credit score classification (Poor, Standard, Good). This step aims to enhance model performance by focusing on high-impact features and reducing noise.
2. **To build and compare machine learning models:** Implement Logistic Regression and Random Forest algorithms to predict credit scores based on the preprocessed dataset. Logistic Regression will provide a baseline model with high interpretability, while Random Forest will capture complex patterns and interactions, allowing for a comparative analysis of model performance.
3. **To evaluate model performance comprehensively:** Assess the predictive accuracy, precision, recall, F1-score, and area under the ROC curve (AUC-ROC) of both models. This objective ensures that the models are evaluated on multiple dimensions, balancing accuracy with robustness and fairness in credit score predictions.
4. **To identify key drivers of credit scores:** Analyze the feature importance derived from WOE, IV, and Random Forest models to pinpoint the most influential factors affecting credit scores. This will provide actionable insights for financial institutions to refine their lending criteria and risk assessment processes.
5. **To contribute to fair and transparent credit scoring:** Develop models that minimize bias and promote fairness in credit score predictions by leveraging WOE and IV to standardize feature contributions and ensure that predictions are based on objective financial behaviors rather than demographic biases.

By achieving these objectives, the study aims to bridge the gap between traditional statistical methods and modern machine learning approaches, offering a scalable and interpretable framework for credit score prediction.

III. Scope

The scope of this study is carefully defined to ensure a focused and feasible investigation into credit score prediction using the provided dataset and specified techniques. The key aspects of the scope are outlined below:

1. **Dataset:** The study utilizes a dataset containing 28 features, including demographic information (e.g., age, occupation), financial metrics (e.g., annual income, monthly balance), credit-related behaviors (e.g., credit utilization ratio, payment behavior), and the target variable, credit score (categorized as Poor, Standard, or Good). The dataset is assumed to be representative of a diverse population of credit applicants, covering various income levels, occupations, and credit histories.
2. **Techniques:** The study focuses on the application of WOE and IV for feature preprocessing and Logistic Regression and Random Forest for predictive modeling. WOE and IV will be used to transform and evaluate features, while Logistic Regression and Random Forest will be employed to build classification models. Other machine learning algorithms (e.g., XGBoost, neural networks) are excluded to maintain a manageable scope and emphasize interpretability.
3. **Target Variable:** The study predicts credit scores classified into three categories: Poor, Standard, and Good. The focus is on multi-class classification, with an emphasis on distinguishing between creditworthy and non-creditworthy individuals to support lending decisions.
4. **Evaluation Metrics:** Model performance will be assessed using standard metrics for classification tasks, including accuracy, precision, recall, F1-score, and AUC-ROC. These metrics will provide a comprehensive view of model effectiveness, with a particular focus on minimizing false positives (e.g., incorrectly classifying risky borrowers as creditworthy).
5. **Application Context:** The study is oriented toward credit scoring in the financial sector, with potential applications in banks, credit unions, and fintech companies. The findings are intended to inform risk assessment practices but are not extended to other domains, such as fraud detection or customer segmentation.
6. **Limitations:** The study is constrained by the quality and completeness of the provided dataset. Missing values, outliers, or biases in the data may impact model performance and are addressed through preprocessing techniques (e.g., imputation, outlier detection). Additionally, the study does not explore real-time deployment or integration with existing credit scoring systems, focusing instead on model development and evaluation.
7. **Ethical Considerations:** The study prioritizes fairness and transparency by using WOE and IV to standardize feature contributions and reduce the risk of bias based on demographic factors (e.g., age, occupation). However, ethical implications of model deployment, such as regulatory compliance, are acknowledged but not deeply explored within the scope.

By adhering to this scope, the study ensures a focused investigation that leverages WOE, IV, Logistic Regression, and Random Forest to deliver meaningful insights into credit score prediction while maintaining clarity and feasibility.

B. THEORETICAL BACKGROUND

The development of predictive models for credit scoring is grounded in a combination of statistical techniques, data preprocessing methods, and machine learning algorithms. This section provides a theoretical review of credit scoring, outlines the methodology for preprocessing and modeling using WOE, IV, Logistic Regression, and Random Forest, and discusses the models' relevance to predicting credit scores based on the provided dataset.

I. *Theoretical Review*

Credit scoring is a quantitative approach used by financial institutions to assess the creditworthiness of individuals, predicting the likelihood of timely repayment of loans or credit obligations. The practice originated in the mid-20th century with the development of statistical models to standardize lending decisions, reducing subjectivity and risk. Credit scores, typically categorized into brackets (e.g., Poor, Standard, Good), are derived from historical data on borrowers' financial behaviors, demographic profiles, and credit activities. These scores guide decisions on loan approvals, interest rates, and credit limits, making accurate predictions critical for both lenders and borrowers.

The dataset provided includes a diverse set of features, such as demographic attributes (age, occupation), financial metrics (annual income, monthly balance), and credit-related variables (credit utilization ratio, outstanding debt, payment behavior). These features align with established credit scoring frameworks, which emphasize the importance of repayment history, debt levels, credit mix, and financial stability. However, raw data often contains noise, imbalances, or non-linear patterns, necessitating preprocessing techniques to enhance predictive power.

Weight of Evidence (WOE) and **Information Value (IV)** are cornerstone techniques in credit scoring for transforming and evaluating features. WOE quantifies the relative risk associated with each category or bin of a variable by calculating the logarithm of the ratio of non-events (e.g., good credit outcomes) to events (e.g., poor credit outcomes). This transformation standardizes variables, reduces the impact of outliers, and improves model interpretability. IV, derived from WOE, measures a variable's predictive strength by assessing how well it differentiates between credit score categories. Variables with high IV values are prioritized in modeling, as they contribute significantly to distinguishing between Poor, Standard, and Good credit scores.

Machine learning has transformed credit scoring by enabling models to capture complex, non-linear relationships in high-dimensional datasets. **Logistic Regression**, a traditional statistical method, remains widely used due to its interpretability and effectiveness in binary or ordinal classification tasks, such as credit score prediction. It models the probability of a specific outcome (e.g., Good credit score) based on a linear combination of features, making it suitable for regulatory environments where transparency is paramount. Conversely, **Random Forest**, an ensemble machine learning algorithm, excels in handling non-linear patterns and feature interactions. By constructing multiple decision trees and aggregating their predictions, Random Forest achieves robustness and high predictive accuracy, particularly in datasets with diverse features like the one provided.

The theoretical foundation of this study integrates WOE and IV for feature engineering with Logistic Regression and Random Forest for predictive modeling. This hybrid approach balances interpretability, predictive power, and robustness, addressing the challenges of credit scoring in a data-driven financial landscape.

II. Methodology

The methodology for this study involves a structured pipeline for data preprocessing, feature engineering, model development, and evaluation, tailored to the dataset's 28 features and the target variable (credit score: Poor, Standard, Good). The key steps are outlined below:

1. Data Preprocessing

1.1. Data Cleaning

- Handle missing values (e.g., in annual income, age, credit history) through imputation techniques, such as median imputation for numerical variables and mode imputation for categorical variables, or based on the relationship between columns.
- Handle outliers by using IQR, lower bound and upper bound to clip these outliers of each columns

1.2. Feature Selection

- Exclude non-predictive features, such as ID, Customer_ID, Name, and SSN, which are identifiers rather than predictors. Retain features like credit utilization ratio, outstanding debt, and payment behavior, which are theoretically linked to creditworthiness.

1.3. Encoding Categorical Variables

- Convert categorical features into numerical formats using techniques like one-hot encoding or ordinal encoding, as appropriate, to prepare them for WOE transformation.

2. Feature Engineering with WOE and IV

Feature engineering is a critical step in preparing the dataset for machine learning models, as it transforms raw data into a format that enhances predictive accuracy and model interpretability. The dataset provided contains 28 features, including demographic attributes (e.g., age, occupation), financial metrics (e.g., annual income, monthly balance), credit-related behaviors (e.g., credit utilization ratio, payment behavior), and the target variable, credit score (categorized as Poor, Standard, or Good). To leverage this rich dataset effectively, Weight of Evidence (WOE) and Information Value (IV) are employed to standardize features, reduce noise, and prioritize variables with strong predictive power. This section outlines the detailed process of binning, WOE calculation, IV computation, and feature selection, ensuring that the resulting inputs are optimized for Logistic Regression and Random Forest models.

2.1. Binning

Binning is the process of grouping continuous or categorical variables into discrete categories (bins) to facilitate WOE and IV calculations. This step is essential for handling the dataset's diverse features, which include both numerical (e.g., annual income, credit utilization ratio) and categorical (e.g., occupation, credit mix) variables. Binning simplifies the data structure, mitigates the impact of outliers, and ensures that WOE transformations are meaningful across different feature types.

- **Continuous Variables:**
 - Continuous features, such as Age, Annual_Income, Monthly_Inhand_Salary, Outstanding_Debt, Credit_Utilization_Ratio, Credit_History_Age, Total_EMI_per_month, Amount_invested_monthly, and Monthly_Balance, are binned into discrete intervals. The binning strategy depends on the feature's distribution and domain relevance:
 - **Quantile-Based Binning:** For features with skewed distributions, such as Annual_Income or Outstanding_Debt, quantiles (e.g., deciles or quartiles) are used to create bins with roughly equal numbers of observations. For example, Annual_Income might be divided into 10 bins, each containing approximately 10% of the data, ensuring balanced representation across income levels.
 - **Domain-Based Binning:** For features with meaningful thresholds, such as Age or Credit_History_Age, bins are defined based on industry standards or logical cutoffs. For instance, Age could be binned into categories like 18–25, 26–35, 36–50, and >50 to reflect life stages relevant to credit behavior. Similarly, Credit_History_Age might be binned into <1 year, 1–3 years, 3–5 years, and >5 years to capture the impact of credit tenure.
 - **Outlier Handling:** Extreme values (e.g., unusually high Outstanding_Debt) are isolated into separate bins to prevent distortion of WOE calculations. For example, values beyond the 99th percentile may be grouped into a single “extreme” bin.
 - The number of bins is chosen to balance granularity and stability, typically ranging from 5 to 10, depending on the feature's variability and sample size. Over-binning is avoided to prevent sparse bins with insufficient data for reliable WOE estimation.
- **Categorical Variables:**
 - Categorical features, such as Occupation, Type_of_Loan, Credit_Mix, Payment_of_Min_Amount, and Payment_Behaviour, are retained in their original categories unless they contain rare or sparse values. For instance:
 - Occupation (e.g., teacher, engineer, entrepreneur) is used as-is, as each category is expected to have sufficient observations.
 - Type_of_Loan, which may include multiple loan types per customer (e.g., auto loan, personal loan), is preprocessed to create a simplified categorization. For example, loan types can be grouped into broader categories like “secured” (e.g., mortgage, auto loan) and “unsecured” (e.g., personal loan, credit card loan) to reduce cardinality.
 - Sparse categories (e.g., rare occupations or loan types with <1% frequency) are combined into an “Other” category to ensure statistical reliability during WOE calculation.

- For ordinal categorical variables, such as Credit_Mix (e.g., Bad, Standard, Good), the natural order is preserved to reflect their inherent ranking.
- **Special Considerations:**
 - Features like Month are evaluated for relevance. Since Month may reflect temporal patterns (e.g., seasonal credit behavior), it is binned into quarters (e.g., Q1: January–March, Q2: April–June) to test its predictive power. If IV analysis indicates low predictiveness, Month may be excluded.
 - Features with high cardinality, such as Type_of_Loan, are carefully grouped to avoid excessive bins, which could lead to overfitting or unstable WOE values.
 - Missing values are treated as a separate bin (e.g., “Missing”) during initial WOE calculation to assess their impact on credit scores. If missingness is predictive, this bin is retained; otherwise, missing values are imputed prior to binning.

Binning ensures that all features are transformed into a format suitable for WOE and IV analysis, creating a consistent framework for comparing predictive power across variables.

2.2. WOE Calculation

Weight of Evidence (WOE) transforms binned features into a logarithmic scale that quantifies the relative risk of each bin with respect to the target variable (credit score: Poor, Standard, Good). WOE is particularly valuable in credit scoring because it standardizes features, enhances model interpretability, and aligns with the logistic nature of credit risk assessment. The WOE calculation is performed as follows:

- **Definition:**
 - WOE measures the difference in the distribution of “non-events” (desirable outcomes, e.g., Standard or Good credit scores) and “events” (undesirable outcomes, e.g., Poor credit scores) for each bin of a feature. The formula is:

$$WOE_i = \ln \left(\frac{\%Non - Events_i}{\%Events_i} \right)$$

Where:

- $\%Non - Events_i = \frac{\text{Count of Non-Events in bin } i}{\text{Total Non-Events}}$
- $\%Events_i = \frac{\text{Count of Events in bin } i}{\text{Total Events}}$
- For this study, “Non-Events” are defined as Standard or Good credit scores, and “Events” are Poor credit scores, reflecting the goal of identifying risky borrowers.
- **Application to Features:** For each binned feature, the dataset is split into events and non-events based on the credit score. For example:
 - **Age:** Suppose Age is binned into 18–25, 26–35, 36–50, and >50. For the 18–25 bin, calculate the proportion of Poor credit scores (events) and Standard/Good credit scores (non-events). If 15% of Poor scores and 5% of Standard/Good scores fall in this bin, the WOE is:

$$WOE_{18-25} = \ln \frac{0.05}{0.15} = \ln 0.333 \approx -1.099$$

A negative WOE indicates that the 18–25 age group is riskier than average, as it has a higher proportion of Poor scores.

- **Credit_Utilization_Ratio:** If binned into <20%, 20–40%, 40–60%, and >60%, a bin like >60% might have a high WOE (e.g., positive value) if it contains a low proportion of Poor scores, indicating lower risk.
- **Payment_Behaviour:** For categories like “High_spent_Small_value_payments” or “Low_spent_Large_value_payments,” WOE reflects how payment patterns correlate with credit scores. A category associated with timely payments may have a positive WOE, signaling lower risk.
- Categorical features follow the same logic. For Occupation, if “Entrepreneur” has a higher proportion of Standard/Good scores than Poor scores, its WOE will be positive, indicating a favorable credit profile.
- **Handling Edge Cases:**
 - **Zero Counts:** If a bin contains no events or non-events (e.g., no Poor scores in a specific Credit_History_Age bin), the WOE becomes undefined due to division by zero. To address this, a small constant (e.g., 0.5) is added to the counts (Laplace smoothing) to ensure numerical stability:

$$\%Events_i = \frac{\text{Count of Events in bin } i + 0.5}{\text{Total Events} + 0.5 \times \text{Number of bins}}$$

- **Missing Values:** Bins labeled “Missing” are assigned WOE values based on their event/non-event distribution. If missingness is non-predictive, imputed values are used instead, and WOE is recalculated.
- **Monotonicity:** WOE values are inspected to ensure monotonic trends where applicable. For example, Credit_History_Age should show increasing WOE (lower risk) as credit history lengthens. Non-monotonic bins may be merged to enforce logical trends.
- **Transformation:**
 - After calculating WOE for each bin, the original feature values are replaced with their corresponding WOE values. For instance, an Annual_Income value falling in the \$50,000–\$75,000 bin is replaced with the WOE of that bin (e.g., 0.75). This transformation standardizes features, aligns them with the log-odds framework of Logistic Regression, and improves the robustness of Random Forest by reducing sensitivity to scale.

WOE transformation creates a dataset where each feature reflects its contribution to credit risk, making it easier for models to distinguish between credit score categories.

2.3. IV Calculation

Information Value (IV) quantifies the predictive power of each feature by measuring how well it differentiates between events (Poor credit scores) and non-events (Standard/Good

credit scores). IV is derived from WOE and serves as a feature selection tool, guiding the inclusion of variables in the final models.

- **Definition:**

- IV is calculated for each feature as the sum of the differences in event and non-event distributions, weighted by WOE:

$$IV = \sum_i (\%Non - Events_i - \%Events_i) \times WOE_i$$

where i represents each bin or category of the feature

IV values are interpreted using standard thresholds:

- < 0.02: Not predictive
- 0.02–0.1: Weakly predictive
- 0.1–0.3: Moderately predictive
- 0.3–0.5: Strongly predictive

- **Application to Features:**

- For each feature, IV is computed based on its WOE values. For example:
 - **Outstanding_Debt:** Suppose the feature is binned into <500, 500–1000, 1000–2000, and >2000 USD. If high-debt bins (>2000) have a high proportion of Poor scores (events) and low-debt bins (<500) have a high proportion of Standard/Good scores (non-events), the IV will be high, reflecting strong predictiveness. An IV of 0.35 would indicate that Outstanding_Debt is a key driver of credit scores.
 - **Credit_History_Age:** Longer credit histories are expected to correlate with better scores. If bins like >5 years have positive WOE and low event rates, while <1 year has negative WOE and high event rates, the IV may be moderately high (e.g., 0.25), confirming its importance.
 - **Occupation:** If certain occupations (e.g., lawyer, doctor) have distinct WOE patterns compared to others (e.g., freelancer), the IV may be moderate (e.g., 0.15), suggesting some predictive value. Low IV (e.g., <0.02) would lead to exclusion unless domain knowledge justifies retention.
- Features like Month are tested for IV to determine if temporal patterns contribute to credit scores. A low IV (e.g., 0.01) would indicate that Month is not predictive and can be dropped.

- **Feature Selection:**

- Features are ranked by IV to prioritize those with the greatest impact on credit score prediction. A threshold of $IV > 0.02$ is typically applied to retain predictive variables, but this is adjusted based on model performance:
 - **High-IV Features** (e.g., Outstanding_Debt, Credit_Utilization_Ratio, Credit_History_Age): These are retained as primary inputs due to their strong association with credit scores.

- **Moderate-IV Features** (e.g., Num_of_Loan, Payment_Behaviour): These are included but monitored for redundancy or interaction effects.
 - **Low-IV Features** (e.g., Month, Num_Bank_Accounts): These are candidates for exclusion unless they improve model performance in combination with other features.
- Redundant features (e.g., Annual_Income and Monthly_Inhand_Salary, which may be highly correlated) are evaluated using correlation analysis post-WOE transformation. If redundancy is confirmed, the feature with lower IV is dropped to simplify the model.
- **Edge Cases:**
 - **Overly Predictive Features:** An IV > 0.5 (e.g., for Num_of_Delayed_Payment) raises concerns about data leakage or overfitting. Such features are scrutinized to ensure they reflect genuine patterns rather than artifacts of the data collection process.
 - **Missing Values:** If the “Missing” bin has a high IV, it suggests that missingness itself is predictive (e.g., missing Credit_History_Age may indicate new borrowers). In such cases, missingness is modeled explicitly; otherwise, imputation is preferred to avoid bias.
 - **Class Imbalance:** Since Poor credit scores may be less frequent than Standard/Good scores, IV calculations account for class imbalance by ensuring sufficient event counts in each bin. If imbalance distorts IV, techniques like oversampling or synthetic data generation (e.g., SMOTE) are considered during modeling.

IV analysis produces a prioritized list of features, ensuring that Logistic Regression and Random Forest models focus on variables most relevant to credit score prediction.

2.4. Integration with Machine Learning Models

The WOE-transformed features and IV-based feature selection are seamlessly integrated into the machine learning pipeline:

- **Logistic Regression:**
 - WOE transformation aligns features with the log-odds framework of Logistic Regression, improving model fit and interpretability. Each WOE value represents the log-odds contribution of a bin, making coefficients directly interpretable as risk adjustments.
 - IV guides feature selection to reduce dimensionality, preventing overfitting and ensuring that only predictive variables (e.g., Outstanding_Debt, Credit_Utilization_Ratio) are included.
- **Random Forest:**
 - WOE transformation standardizes features, reducing the impact of scale differences (e.g., between Annual_Income and Num_of_Loan). This enhances Random Forest’s ability to split nodes effectively across diverse variables.

- IV informs feature importance analysis, as high-IV features are expected to rank highly in Random Forest's Gini importance metric, confirming their role in decision trees.
- **Model Optimization:**
 - The WOE-transformed dataset is split into training and testing sets, with IV rankings guiding iterative feature inclusion. For example, models are initially trained with high-IV features, then tested with moderate-IV features to assess incremental gains.
 - Hyperparameter tuning (e.g., regularization for Logistic Regression, tree depth for Random Forest) is performed to optimize performance on WOE-transformed data, ensuring robustness across credit score categories.

2.5. Practical Considerations

- **Interpretability:** WOE values provide a clear interpretation of risk. For instance, a positive WOE for Credit_History_Age >5 years indicates lower risk, which can be communicated to stakeholders in financial institutions.
- **Scalability:** The binning, WOE, and IV calculations are automated using Python libraries (e.g., pandas, numpy), ensuring scalability for large datasets. Custom scripts handle edge cases like zero counts or sparse bins.
- **Fairness:** WOE and IV help mitigate bias by focusing on financial behaviors (e.g., Payment_Behaviour, Outstanding_Debt) rather than demographic factors (e.g., Age, Occupation). Features with potential bias are flagged during IV analysis and evaluated for fairness.
- **Validation:** The stability of WOE and IV values is tested using cross-validation to ensure consistency across data subsets. Significant variations may indicate the need for re-binning or data quality checks.

This feature engineering process transforms the raw dataset into a predictive, standardized format, enabling Logistic Regression and Random Forest to deliver accurate and interpretable credit score predictions.

3. Model Development

Model development is a pivotal phase in the credit score prediction pipeline, where machine learning algorithms are trained to classify individuals into credit score categories (Poor, Standard, Good) based on the provided dataset. The dataset includes 28 features, such as demographic attributes (e.g., age, occupation), financial metrics (e.g., annual income, outstanding debt), and credit-related behaviors (e.g., credit utilization ratio, payment behavior), which have been preprocessed using WOE and IV to enhance predictive power. Two algorithms—**Logistic Regression** and **Random Forest**—are selected for their complementary strengths: Logistic Regression for its interpretability and Random Forest for its ability to capture complex patterns. To optimize model performance, **GridSearchCV** is employed for systematic hyperparameter tuning. This section details the data preparation, model implementation, hyperparameter optimization, and training process, ensuring robust and accurate credit score predictions.

3.1. Data Preparation

Before model training, the dataset is prepared to ensure compatibility with Logistic Regression and Random Forest algorithms. The following steps are undertaken:

- **WOE-Transformed Features:** The raw features are replaced with their WOE values, as calculated in the feature engineering phase. For example, Outstanding_Debt values are binned (e.g., <500, 500–1000, >1000 USD) and assigned WOE scores based on their event (Poor credit score) and non-event (Standard/Good credit score) distributions. This transformation standardizes inputs, aligns with Logistic Regression’s log-odds framework, and reduces Random Forest’s sensitivity to feature scales.
 - **Feature Selection Based on IV:** Features are prioritized using IV rankings. High-IV features (e.g., Outstanding_Debt, Credit_Utilization_Ratio, Credit_History_Age, $IV > 0.3$) are retained as primary predictors, while moderate-IV features (e.g., Num_of_Loan, Payment_Behaviour, $IV 0.1–0.3$) are included conditionally. Low-IV features (e.g., Month, $IV < 0.02$) are excluded unless they enhance performance in combination with others. This reduces dimensionality, mitigates overfitting, and focuses models on predictive variables.
 - **Data Splitting:** The dataset is divided into training (80%) and testing (20%) sets using stratified sampling to maintain the proportion of credit score categories (e.g., 10% Poor, 50% Standard, 40% Good). This ensures that both sets reflect the class distribution, critical for evaluating performance on the minority Poor class. A validation set is not explicitly carved out, as GridSearchCV uses cross-validation internally for hyperparameter tuning.
 - **Handling Class Imbalance:** The Poor class may be underrepresented, posing a challenge for model training. To address this:
 - **Class Weights:** Both models are configured to assign higher weights to the Poor class, emphasizing correct classification of risky borrowers.
 - **Oversampling Consideration:** Techniques like Synthetic Minority Oversampling Technique (SMOTE) are tested during experimentation, but applied cautiously to avoid introducing synthetic noise into WOE-transformed features.
 - **Scaling and Normalization:** Since WOE transformation standardizes features to a logarithmic scale, additional scaling (e.g., Min-Max or StandardScaler) is unnecessary for Random Forest, which is scale-invariant. For Logistic Regression, WOE values are inherently normalized, but multicollinearity is checked using variance inflation factors (VIF) to ensure stable coefficients.
- The prepared dataset, consisting of WOE-transformed features and a stratified train-test split, serves as the foundation for model development.

3.2. Logistic Regression

Overview: Logistic Regression is a generalized linear model that predicts the probability of categorical outcomes, making it well-suited for multi-class credit score prediction (Poor, Standard, Good). In this study, multinomial Logistic Regression is used to model the log-odds of each class relative to a reference category (e.g., Good), leveraging WOE-transformed features for enhanced interpretability and performance.

Implementation:

- **Model Specification:** The Logistic Regression model is implemented using Python’s scikit-learn library (`sklearn.linear_model.LogisticRegression`). The `multi_class='multinomial'` parameter enables softmax regression, where the probability of class k (e.g., Poor) is:

$$P(y = k|X) = \frac{\exp(\beta_k^T X)}{\sum_{j=1}^K \exp(\beta_j^T X)}$$

Here, X is the WOE-transformed feature vector, β_k are class-specific coefficients, and $K = 3$ (Poor, Standard, Good).

- **Regularization:** To prevent overfitting, L2 regularization (Ridge penalty) is applied, controlled by the hyperparameter C (inverse of regularization strength). A smaller C increases regularization, shrinking coefficients to reduce model complexity.
- **Class Weighting:** The `class_weight='balanced'` option adjusts weights inversely proportional to class frequencies, assigning higher penalties to misclassifications of the Poor class. Alternatively, custom weights (e.g., Poor: 5, Standard: 1, Good: 1) are tested to prioritize risk identification.
- **Solver:** The `solver='lbfgs'` option is used for optimization, as it supports multinomial loss and converges efficiently for small to medium datasets. For larger datasets, `saga` is considered to handle scalability.

Training Process:

- The model is trained on the WOE-transformed training set, fitting coefficients to maximize the log-likelihood of observed credit scores. For example, a positive coefficient for `Credit_History_Age`’s WOE indicates that longer credit histories reduce the log-odds of a Poor score.
- Convergence is monitored using a maximum iteration limit (`max_iter=1000`) to ensure stable coefficient estimates, particularly given the dataset’s dimensionality after feature selection.
- Coefficients are inspected post-training to verify alignment with credit scoring theory (e.g., negative coefficients for high WOE values of `Outstanding_Debt`, indicating higher risk).

Rationale:

- Logistic Regression is chosen for its interpretability, as coefficients directly reflect the impact of WOE-transformed features on credit scores. This is critical for regulatory compliance in financial applications, where stakeholders require transparent risk assessments.
- WOE transformation enhances Logistic Regression’s performance by linearizing relationships and reducing multicollinearity, making it a strong baseline for comparison with Random Forest.

3.3. Random Forest

Overview: Random Forest is an ensemble learning method that constructs multiple decision trees, each trained on a random subset of data and features, and aggregates their predictions to produce robust classifications. Its ability to model non-linear relationships and feature interactions makes it ideal for the complex patterns in the credit score dataset.

Implementation:

- **Model Specification:** The Random Forest classifier is implemented using `sklearn.ensemble.RandomForestClassifier`. Key parameters include:
 - `n_estimators`: Number of trees (e.g., 100–500), balancing accuracy and computational cost.

- **max_depth**: Maximum tree depth to control overfitting (e.g., 10–30).
- **min_samples_split** and **min_samples_leaf**: Minimum samples required to split a node or form a leaf, preventing overly specific trees.
- **max_features**: Number of features considered at each split (e.g., 'sqrt' or a fraction of total features), enhancing diversity.
- **Class Weighting**: Similar to Logistic Regression, `class_weight='balanced'` or custom weights (e.g., Poor: 5, Standard: 1, Good: 1) are applied to prioritize the Poor class, ensuring sensitivity to risky borrowers.
- **Bootstrap Sampling**: Random Forest uses bootstrapping (`bootstrap=True`) to create diverse training subsets for each tree, reducing variance and improving generalization.
- **Random Feature Selection**: At each node, a random subset of features is evaluated, leveraging WOE-transformed inputs like `Credit_Utilization_Ratio` and `Payment_Behaviour` to uncover predictive patterns.

Training Process:

- The model is trained on the training set, with each tree learning decision rules based on WOE-transformed features. For example, a tree might split on `Outstanding_Debt`'s WOE at a threshold that separates Poor from Standard/Good scores.
- Predictions are aggregated via majority voting, where each tree votes for a credit score category, and the class with the most votes is selected.
- Feature importance is calculated using Gini impurity reduction, highlighting key predictors (e.g., `Outstanding_Debt`, `Credit_History_Age`) for post-hoc analysis.

Rationale:

- Random Forest is selected for its robustness to noise, ability to handle high-dimensional data, and superior performance in imbalanced settings. Its ensemble nature mitigates overfitting, making it suitable for the diverse feature set.
- WOE transformation ensures that Random Forest focuses on risk-relevant patterns, while IV-based feature selection reduces computational overhead by eliminating low-value features.

3.4. GridSearchCV for Hyperparameter Tuning

Overview: GridSearchCV is a systematic approach to hyperparameter optimization, evaluating multiple parameter combinations via cross-validation to identify the configuration that maximizes model performance. It is applied to both Logistic Regression and Random Forest to ensure optimal predictive accuracy and robustness.

Implementation:

- **Cross-Validation Setup**: GridSearchCV uses 5-fold cross-validation, dividing the training set into five subsets. For each fold, the model is trained on four subsets and validated on the fifth, averaging performance across folds to estimate generalization error.
- **Performance Metric**: The primary metric for optimization is the macro-average F1-Score, as it balances precision and recall across all classes, with emphasis on the Poor class. Alternative metrics (e.g., accuracy, AUC-ROC) are monitored to ensure comprehensive evaluation.
- **Logistic Regression Grid Search:**

- **Parameters:**
 - C: Regularization strength, tested over a range (e.g., [0.001, 0.01, 0.1, 1, 10, 100]). Lower values increase regularization, reducing overfitting but potentially underfitting.
 - class_weight: Options include 'balanced' and custom weights (e.g., {0: 5, 1: 1, 2: 1} for Poor, Standard, Good), optimizing for Poor class performance.
 - solver: Tested with 'lbfgs' and 'saga' to ensure convergence for multinomial loss.
- **Search Process:** GridSearchCV evaluates all combinations (e.g., 6 C values $\times$ 2 class_weight options $\times$ 2 solver options = 24 configurations) using 5-fold cross-validation, selecting the configuration with the highest macro-F1.
- **Example:** A configuration with C=0.1, class_weight='balanced', and solver='lbfgs' might yield a macro-F1 of 0.78, indicating balanced performance.
- **Random Forest Grid Search:**
 - **Parameters:**
 - n_estimators: Number of trees, tested over [100, 200, 300, 500].
 - max_depth: Tested over [10, 20, 30, None] (None allows unlimited depth).
 - min_samples_split: Tested over [2, 5, 10].
 - min_samples_leaf: Tested over [1, 2, 4].
 - max_features: Tested over ['sqrt', 'log2', 0.5] (0.5 uses 50% of features).
 - class_weight: Options include 'balanced' and custom weights (e.g., {0: 5, 1: 1, 2: 1}).
 - **Search Process:** GridSearchCV evaluates a subset of combinations (e.g., 4 n_estimators $\times$ 4 max_depth $\times$ 3 min_samples_split $\times$ 3 min_samples_leaf $\times$ 3 max_features $\times$ 2 class_weight = 864 configurations) to manage computational cost. To reduce runtime, a coarse grid is tested first, followed by a finer grid around promising values.
 - **Example:** A configuration with n_estimators=200, max_depth=20, min_samples_split=5, min_samples_leaf=2, max_features='sqrt', and class_weight='balanced' might achieve a macro-F1 of 0.85, reflecting strong performance.
- **Computational Considerations:**
 - GridSearchCV is computationally intensive, particularly for Random Forest with large parameter grids. To optimize efficiency:
 - **Parallel Processing:** The n_jobs=-1 parameter leverages all available CPU cores to parallelize cross-validation.
 - **Randomized Search:** For Random Forest, RandomizedSearchCV is tested as an alternative, sampling 100 random combinations to approximate the optimal configuration.
 - **Early Stopping:** If a parameter range (e.g., C > 10 for Logistic Regression) consistently underperforms, it is excluded from further searches.
 - **Caching:** Intermediate results are cached to avoid redundant computations during iterative tuning.
- **Output:**
 - For each model, GridSearchCV returns the best hyperparameters, cross-validated macro-F1 score, and performance metrics for alternative configurations. For example:

- Logistic Regression: Best parameters ($C=0.1$, $\text{class_weight}=\text{'balanced'}$, $\text{solver}=\text{'lbfgs'}$) with macro-F1 = 0.78.
- Random Forest: Best parameters ($n_estimators=200$, $\text{max_depth}=20$, etc.) with macro-F1 = 0.85.
- The best models are retrained on the entire training set using the optimal hyperparameters, preparing them for evaluation on the test set.

Rationale:

- GridSearchCV ensures that both models are fine-tuned to maximize performance, particularly for the Poor class, where errors are costly. By systematically exploring parameter spaces, it mitigates the risk of suboptimal configurations.
- The macro-F1 metric aligns with the project's goal of balanced performance across classes, while cross-validation ensures robustness to data variability.

3.5. Training and Validation

Training:

- **Logistic Regression:** The model is trained using the best hyperparameters from GridSearchCV. The training process is lightweight, typically converging within seconds due to the WOE-transformed dataset's reduced dimensionality (e.g., 10–15 high-IV features). Coefficients are saved for interpretability, enabling stakeholders to understand feature impacts (e.g., negative coefficient for `Credit_Utilization_Ratio`'s WOE indicates higher risk).
- **Random Forest:** Training is more computationally intensive, requiring minutes to hours depending on $n_estimators$ and dataset size. The model is trained in parallel to expedite processing, with tree diversity ensured by bootstrap sampling and random feature selection.

Validation:

- **Internal Validation:** During GridSearchCV, 5-fold cross-validation provides an initial estimate of model performance. For example, a cross-validated macro-F1 of 0.78 for Logistic Regression and 0.85 for Random Forest indicates Random Forest's superior generalization.
- **Overfitting Check:** Training performance (e.g., macro-F1 on training set) is compared to cross-validation scores. Significant gaps (e.g., training F1 = 0.95, CV F1 = 0.85) prompt stronger regularization (C for Logistic Regression, max_depth for Random Forest).
- **Feature Importance:** Post-training, Logistic Regression coefficients and Random Forest's Gini importance are analyzed to confirm that high-IV features (e.g., `Outstanding_Debt`, `Credit_History_Age`) dominate predictions, validating the WOE/IV pipeline.

Iterative Refinement:

- If initial models underperform (e.g., low recall for Poor), the following adjustments are tested:
 - Revisit feature selection to include moderate-IV features excluded earlier.
 - Adjust class weights to further emphasize the Poor class.
 - Experiment with ensemble methods (e.g., combining Logistic Regression and Random Forest predictions) to boost performance.
- The process iterates until models achieve satisfactory cross-validated performance, balancing accuracy, interpretability, and computational efficiency.

3.6. Practical Considerations

- **Interpretability:** Logistic Regression's coefficients are documented to explain risk drivers (e.g., high WOE for Payment_Behaviour reduces Poor score probability), supporting regulatory compliance. Random Forest's feature importance rankings provide complementary insights, though less granular.
- **Scalability:** Logistic Regression scales well to large datasets, while Random Forest's runtime is optimized through parallelization and parameter pruning. Both models are feasible for the dataset's expected size (e.g., thousands to tens of thousands of records).
- **Fairness:** WOE and IV preprocessing minimizes bias by focusing on financial behaviors rather than demographics. Models are monitored for disparate performance across groups (e.g., age, occupation) to ensure equitable predictions.
- **Reproducibility:** Random seeds are set (random_state=42) for consistent results across runs, critical for academic and operational validation.

This model development process produces optimized Logistic Regression and Random Forest classifiers, ready for evaluation on the test set to assess their effectiveness in credit score prediction.

4. Model Evaluation

Model evaluation is a critical phase in the development of machine learning models for credit score prediction, as it quantifies their ability to accurately classify individuals into credit score categories (Poor, Standard, Good) based on the provided dataset. The dataset, comprising 28 features such as demographic attributes (e.g., age, occupation), financial metrics (e.g., annual income, outstanding debt), and credit-related behaviors (e.g., credit utilization ratio, payment behavior), has been preprocessed using WOE and IV to enhance predictive power. Two models—Logistic Regression and Random Forest—are evaluated using a suite of performance metrics: **Accuracy**, **Precision**, **Recall**, **F1-Score**, **Area Under the Receiver Operating Characteristic Curve (AUC-ROC)**, and **Gini Coefficient**. These metrics collectively assess the models' correctness, class-specific performance, discriminative ability, and practical utility in credit risk assessment. This section details the computation, interpretation, and comparative analysis of these metrics, ensuring a robust evaluation framework.

4.1. Evaluation Framework

To ensure a fair and reliable assessment, the dataset is split into training (70%) and testing (30%) sets, with stratified sampling to preserve the distribution of credit score categories (Poor, Standard, Good). This split mitigates bias due to class imbalance, as Poor credit scores may be less frequent than Standard or Good scores. Model performance is evaluated on the test set to measure generalization to unseen data. Additionally, k-fold cross-validation (k=5) is employed to assess model stability across different data subsets, reducing the risk of overfitting and ensuring consistent results.

Each metric is calculated for both Logistic Regression and Random Forest, with results reported for the overall model (macro-average across classes) and per-class (Poor, Standard, Good) to highlight performance variations. The evaluation emphasizes the Poor category, as misclassifying

risky borrowers has significant financial implications for lenders. The metrics are computed using standard Python libraries (e.g., scikit-learn), ensuring precision and reproducibility.

4.2. Accuracy

Definition: Accuracy measures the proportion of correct predictions across all credit score categories. It is calculated as:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

where correct predictions are instances where the predicted credit score matches the actual score (Poor, Standard, or Good).

- For Random Forest, which may capture complex patterns more effectively due to its ensemble nature, a higher accuracy (e.g., 85%) is expected, reflecting its ability to handle non-linear relationships in features like Credit_Utilization_Ratio and Payment_Behaviour.
- **Per-Class Insight:** Accuracy is broken down by class to identify imbalances. For instance, if the Poor category has fewer instances (e.g., 300 vs. 1,500 for Standard and 1,200 for Good), a high overall accuracy may mask poor performance on the Poor class. Class-specific accuracy is computed as the proportion of correct predictions within each category.

Interpretation:

- A high accuracy (>80%) indicates robust overall performance, but it must be interpreted cautiously in the presence of class imbalance. For credit scoring, overpredicting Standard or Good scores at the expense of Poor scores can inflate accuracy while increasing financial risk.
- Logistic Regression, with its linear assumptions, may achieve moderate accuracy (e.g., 75–80%) due to its reliance on WOE-transformed features. Random Forest, leveraging feature interactions, is likely to outperform it (e.g., 80–90%).
- **Limitations:** Accuracy alone is insufficient, as it does not distinguish between types of errors (e.g., false positives vs. false negatives). Thus, it is complemented by class-specific metrics.

4.3. Precision

Definition: Precision measures the proportion of predicted instances for a given class that are correctly classified. It is calculated as:

$$\text{Precision}_k = \frac{\text{True Positives}_k}{\text{True Positives}_k + \text{False Positives}_k}$$

where k represents the credit score class (Poor, Standard, Good), True Positives (TP) are correct predictions for class k , and False Positives (FP) are incorrect predictions for class k .

- **Macro-Average Precision:** The average precision across all classes is computed to assess overall reliability:

$$\text{Macro - Precision} = \frac{\text{Precision}_{\text{Poor}} + \text{Precision}_{\text{Standard}} + \text{Precision}_{\text{Good}}}{3}$$

- **Per-Class Insight:** Precision is evaluated for each class:
 - **Poor:** High precision is prioritized to avoid false positives (e.g., labeling a Standard borrower as Poor), which could lead to unfair denials.
 - **Standard and Good:** Moderate precision is acceptable, as errors in these classes are less costly but still impact customer trust and operational efficiency.

Interpretation:

- Logistic Regression may achieve high precision for Standard and Good classes (e.g., 85%) due to their prevalence but lower precision for Poor (e.g., 70%) if it struggles with minority class patterns.
- Random Forest is expected to yield higher precision across all classes (e.g., 80% for Poor, 90% for Standard/Good), leveraging its ability to model complex relationships in WOE-transformed features.
- **Trade-Offs:** High precision often comes at the cost of recall, particularly for the Poor class, necessitating a balanced evaluation with other metrics.

4.4. Recall

Definition: Recall (or sensitivity) measures the proportion of actual instances of a given class that are correctly identified. It is calculated as:

$$Recall_k = \frac{True\ Positives_k}{True\ Positives_k + False\ Negatives_k}$$

where False Negatives (FN) are instances of class k incorrectly predicted as another class.

- **Macro-Average Recall:** The average recall across all classes provides a holistic view:

$$Macro - Recall = \frac{Recall_{Poor} + Recall_{Standard} + Recall_{Good}}{3}$$

- **Per-Class Insight:**
 - **Poor:** High recall is critical to ensure that most risky borrowers are flagged, even if it increases false positives.
 - **Standard and Good:** Lower recall is tolerable, as misclassifying these classes has less severe consequences, though it affects customer segmentation.

Interpretation:

- Logistic Regression may exhibit lower recall for Poor (e.g., 65–75%) due to its linear nature, potentially missing nuanced patterns in features like Num_of_Delayed_Payment.
- Random Forest is expected to achieve higher recall (e.g., 80–90% for Poor) by capturing interactions among features like Outstanding_Debt and Credit_History_Age.
- **Trade-Offs:** High recall for Poor may reduce precision, leading to more false positives. The balance is assessed using F1-Score.

4.5. F1-Score

Definition: The F1-Score is the harmonic mean of precision and recall, providing a balanced measure of a model's performance for each class. It is calculated as:

$$F1 - Score_k = 2 \times \frac{Precision_k \times Recall_k}{Precision_k + Recall_k}$$

- **Per-Class Insight:**
 - **Poor:** A high F1-Score ensures that the model balances identifying risky borrowers with minimizing false positives, critical for risk management.
 - **Standard and Good:** High F1-Scores indicate reliable classification, supporting accurate customer segmentation.

Interpretation:

- Logistic Regression may achieve moderate F1-Scores (e.g., 70% for Poor, 80% for Standard/Good), reflecting its limitations with minority classes.
- Random Forest is likely to outperform with higher F1-Scores (e.g., 80–85% for Poor, 85–90% for Standard/Good), benefiting from WOE transformations and ensemble learning.
- **Use Case:** The F1-Score guides model selection when both precision and recall are critical, particularly for the Poor class, where errors are costly.

4.6 AUC-ROC

Definition: The Area Under the Receiver Operating Characteristic Curve (AUC-ROC) measures a model's ability to distinguish between classes. For multi-class classification, AUC-ROC is computed using one-vs-rest (OvR) or one-vs-one (OvO) approaches:

- **OvR:** For each class (e.g., Poor), the model's ability to separate it from all others is evaluated, producing a class-specific AUC.
- **OvO:** Pairwise comparisons (e.g., Poor vs. Standard, Poor vs. Good) are averaged. The ROC curve plots the True Positive Rate (Recall) against the False Positive Rate at various thresholds, and AUC quantifies the overall discriminative power.
- **Per-Class Insight:**
 - **Poor:** A high AUC-ROC (>0.85) indicates strong separation of risky borrowers, crucial for minimizing default risk.
 - **Standard and Good:** High AUCs ensure reliable differentiation, supporting accurate credit approvals.

Interpretation:

- Logistic Regression may achieve moderate AUC-ROC (e.g., 0.80–0.85 for Poor), limited by its linear assumptions but improved by WOE transformations.
- Random Forest is expected to yield higher AUC-ROC (e.g., 0.85–0.95 for Poor), reflecting its ability to model complex patterns in features like Credit_Mix and Delay_from_due_date.
- **Advantages:** AUC-ROC is threshold-independent, making it robust to class imbalance and suitable for comparing models across different datasets.
- **Limitations:** AUC-ROC may overestimate performance in imbalanced settings, necessitating validation with precision, recall, and F1-Score.

4.7. Gini Coefficient

Definition: The Gini Coefficient, derived from AUC-ROC, measures a model's discriminative ability, particularly in credit scoring contexts. It is calculated as:

$$Gini = 2 \times AUC - 1$$

For multi-class problems, Gini is computed per class (using OvR AUCs) or as a macro-average.

- **Per-Class Insight:**
 - **Poor:** A high Gini (>0.70) indicates strong identification of risky borrowers, aligning with credit risk objectives.
 - **Standard and Good:** High Gini values ensure accurate classification, supporting operational efficiency.

Interpretation:

- Logistic Regression may achieve moderate Gini values (e.g., 0.60–0.70 for Poor), reflecting its reliance on WOE-transformed features.
- Random Forest is expected to produce higher Gini values (e.g., 0.70–0.90 for Poor), leveraging its ensemble approach to maximize separation.
- **Use Case:** The Gini Coefficient is widely used in credit scoring to quantify model quality, as it directly relates to the economic value of accurate predictions.
- **Limitations:** Like AUC-ROC, Gini may be less informative in highly imbalanced datasets, requiring integration with other metrics.

4.8. Comparative Analysis

The evaluation metrics are compared across Logistic Regression and Random Forest to identify the optimal model:

- **Accuracy:** Random Forest is likely to outperform Logistic Regression due to its ability to capture non-linear patterns, but both models are assessed for class-specific performance.
- **Precision, Recall, F1-Score:** Random Forest is expected to excel in the Poor class, balancing precision and recall for risk identification. Logistic Regression provides a baseline, with potential weaknesses in minority class performance.
- **AUC-ROC and Gini:** Random Forest's higher values reflect superior discriminative ability, while Logistic Regression's performance is enhanced by WOE transformations, making it competitive in interpretable settings.
- **Feature Importance:** Logistic Regression coefficients and Random Forest's Gini importance are analyzed to validate the role of high-IV features (e.g., Outstanding_Debt, Credit_History_Age), ensuring alignment with credit scoring theory.

C. DATA

I. Data Description

	Column name	Description
1	ID	Represents a unique identification of an entry
2	Customer_ID	Represents a unique identification of a person
3	Month	Represents the month of the year
4	Name	Represents the name of a person
5	Age	Represents the age of the person
6	SSN	Represents the social security number of a person
7	Occupation	Represents the occupation of the person
8	Annual_Income	Represents the annual income of the person
9	Monthly_Inhand_Salary	Represents the monthly base salary of a person
10	Num_Bank_Accounts	Represents the number of bank accounts a person holds
11	Num_Credit_Card	Represents the number of other credit cards held by a person
12	Interest_Rate	Represents the interest rate on credit card
13	Num_of_Loan	Represents the number of loans taken from the bank
14	Type_of_Loan	Represents the types of loan taken by a person
15	Delay_from_due_date	Represents the average number of days delayed from the payment date
16	Num_of_Delayed_Payment	Represents the average number of payments delayed by a person
17	Changed_Credit_Limit	Represents the percentage change in credit card limit
18	Num_Credit_Inquiries	Represents the number of credit card inquiries
19	Credit_Mix	Represents the classification of the mix of credits
20	Outstanding_Debt	Represents the remaining debt to be paid (in USD)
21	Credit_Utilization_Ratio	Represents the utilization ratio of credit card
22	Credit_History_Age	Represents the age of credit history of the person
23	Payment_of_Min_Amount	Represents whether only the minimum amount was paid by the person
24	Total_EMI_per_month	Represents the monthly EMI payments (in USD)
25	Amount_invested_monthly	Represents the monthly amount invested by the customer (in USD)

26	Payment_Behaviour	Represents the payment behavior of the customer (in USD)
27	Monthly_Balance	Represents the monthly balance amount of the customer (in USD)
28	Credit_Score	Represents the bracket of credit score (Poor, Standard, Good)

Data shape: (100000, 28)

II. Data Preprocessing and Exploratory Data Analysis

1. Identification of Numeric and Categorical Columns

The first segment of the code identifies the numeric and categorical columns in the training dataset (df_train) to understand the data's structure and guide subsequent preprocessing steps.

```
numeric_cols = df_train.select_dtypes(include=['int64', 'float64']).columns.tolist()
categoric_cols = df_train.select_dtypes(include=['object', 'category']).columns.tolist()

print("Numeric columns:")
print(numeric_cols)
print("\nCategorical columns:")
print(categoric_cols)
```

Output:

Numeric columns:

['Monthly_Inhand_Salary', 'Num_Bank_Accounts', 'Num_Credit_Card', 'Interest_Rate', 'Delay_from_due_date', 'Num_Credit_Inquiries', 'Credit_Utilization_Ratio', 'Total_EMI_per_month']

Categorical columns:

['ID', 'Customer_ID', 'Month', 'Name', 'Age', 'SSN', 'Occupation', 'Annual_Income', 'Num_of_Loan', 'Type_of_Loan', 'Num_of_Delayed_Payment', 'Changed_Credit_Limit', 'Credit_Mix', 'Outstanding_Debt', 'Credit_History_Age', 'Payment_of_Min_Amount', 'Amount_invested_monthly', 'Payment_Behaviour', 'Monthly_Balance', 'Credit_Score']

Explanation:

- **Purpose:** The code uses pandas' select_dtypes method to categorize columns based on their data types. Numeric columns are those with int64 or float64 types, suitable for mathematical operations, while categorical columns include object or category types, typically representing text or encoded values.
- **Findings:**
 - **Numeric Columns:** Eight features, such as Monthly_Inhand_Salary, Credit_Utilization_Ratio, and Total_EMI_per_month, are correctly identified as numeric, aligning with their expected continuous or discrete nature. These features

are ready for operations like binning for WOE calculation without further type conversion.

- **Categorical Columns:** Twenty features, including Annual_Income, Outstanding_Debt, Age, and Credit_Score, are classified as categorical, indicating they are stored as strings (object) or categories. This is unexpected for features like Annual_Income and Outstanding_Debt, which are inherently numeric but likely contain non-numeric characters (e.g., commas, dollar signs) or inconsistencies (e.g., empty strings).
- **Implications:** The presence of numeric features in the categorical list highlights data quality issues that must be resolved to enable quantitative analysis. For example, Age and Annual_Income need to be converted to numeric formats to support binning and WOE transformation, critical for Logistic Regression and Random Forest modeling.
- **Role in Project:** Identifying column types informs the preprocessing strategy, ensuring that numeric features are treated appropriately for statistical operations and categorical features are encoded or cleaned as needed. This step is crucial for preparing the dataset for feature engineering, where WOE and IV will quantify the predictive power of features like Credit_Utilization_Ratio and Outstanding_Debt.
- **Significance:**
 - Accurate identification of feature types prevents errors in downstream processes, such as applying numeric transformations to text data or misinterpreting categorical variables as continuous.
 - The output reveals that several key predictors (e.g., Annual_Income, Outstanding_Debt) require type conversion, setting the stage for the next preprocessing steps.

2. Conversion of Categorical Columns to Numeric

The second segment of the code addresses the issue of numeric features misclassified as categorical by cleaning and converting specific columns to float type.

```

columns_to_convert = [
    "Annual_Income",
    "Changed_Credit_Limit",
    "Outstanding_Debt",
    "Amount_invested_monthly",
    "Monthly_Balance",
    "Age",
    "Num_of_Loan",
    "Num_of_Delayed_Payment"
]

# Replace non-numeric characters and empty strings with NaN, then convert to float
df_train[columns_to_convert] = (
    df_train[columns_to_convert]
    .replace('[^\d.]', '', regex=True) # Remove unwanted characters
    .replace('', np.nan) # Replace empty strings with NaN
    .astype(float) # Convert to float
)

```

Explanation:

- **Purpose:** This code targets eight columns identified as categorical but expected to be numeric based on their semantic meaning (e.g., Annual_Income represents monetary values, Age represents years). The goal is to clean these columns by removing non-numeric characters and converting them to float type.
- **Steps:**
 - **Column Selection:** The list columns_to_convert includes features critical for credit scoring, such as Annual_Income (financial capacity), Outstanding_Debt (debt burden), Age (demographic factor), and Num_of_Delayed_Payment (payment behavior). These features are essential for WOE and IV calculations, requiring numeric formats.
 - **Removing Non-Numeric Characters:** The replace('[^\d.]', '', regex=True) operation uses a regular expression to remove any characters except digits (\d) and decimal points (.). For example, an Annual_Income value like “\$50,000.00” is cleaned to “50000.00”, and “N/A” is reduced to an empty string.
 - **Handling Empty Strings:** The replace('', np.nan) step converts empty strings (resulting from cleaning invalid entries like “--” or “N/A”) to NaN, ensuring that missing values are explicitly marked for further handling.
 - **Type Conversion:** The astype(float) method converts the cleaned values to float, enabling mathematical operations like binning, summation, or WOE computation. The float type accommodates decimal values (e.g., Changed_Credit_Limit as a percentage) and integers (e.g., Num_of_Loan).
- **Outcome:** The eight columns are transformed into numeric format, correcting their misclassification as categorical and enabling quantitative analysis.
- **Significance:**

- Converting these columns to numeric is essential for feature engineering, as WOE requires binning continuous or discrete values (e.g., Annual_Income into ranges like <50,000, 50,000–100,000). Without this step, these features would be unusable for predictive modeling.
- Marking invalid entries as NaN facilitates missing value imputation, a subsequent preprocessing step that ensures data completeness.
- The cleaned features align with credit scoring theory, where Outstanding_Debt and Num_of_Delayed_Payment are expected to strongly influence credit scores, as validated later by IV rankings.
- **Challenges Addressed:**
 - **Data Inconsistencies:** Non-numeric characters (e.g., commas, underscores) are common in real-world datasets due to data entry errors or formatting conventions. The code robustly handles these issues.
 - **Missing Values:** By converting invalid entries to NaN, the code prepares the dataset for imputation strategies (e.g., median for Annual_Income, mode for Num_of_Loan), preserving data integrity.

3. Additional Numeric Conversion with Error Handling

The third segment of the code applies a more cautious conversion for a subset of columns, using `pd.to_numeric` to handle potential errors gracefully.

```
columns_to_convert = ["Age", "Num_of_Loan", "Num_of_Delayed_Payment"]

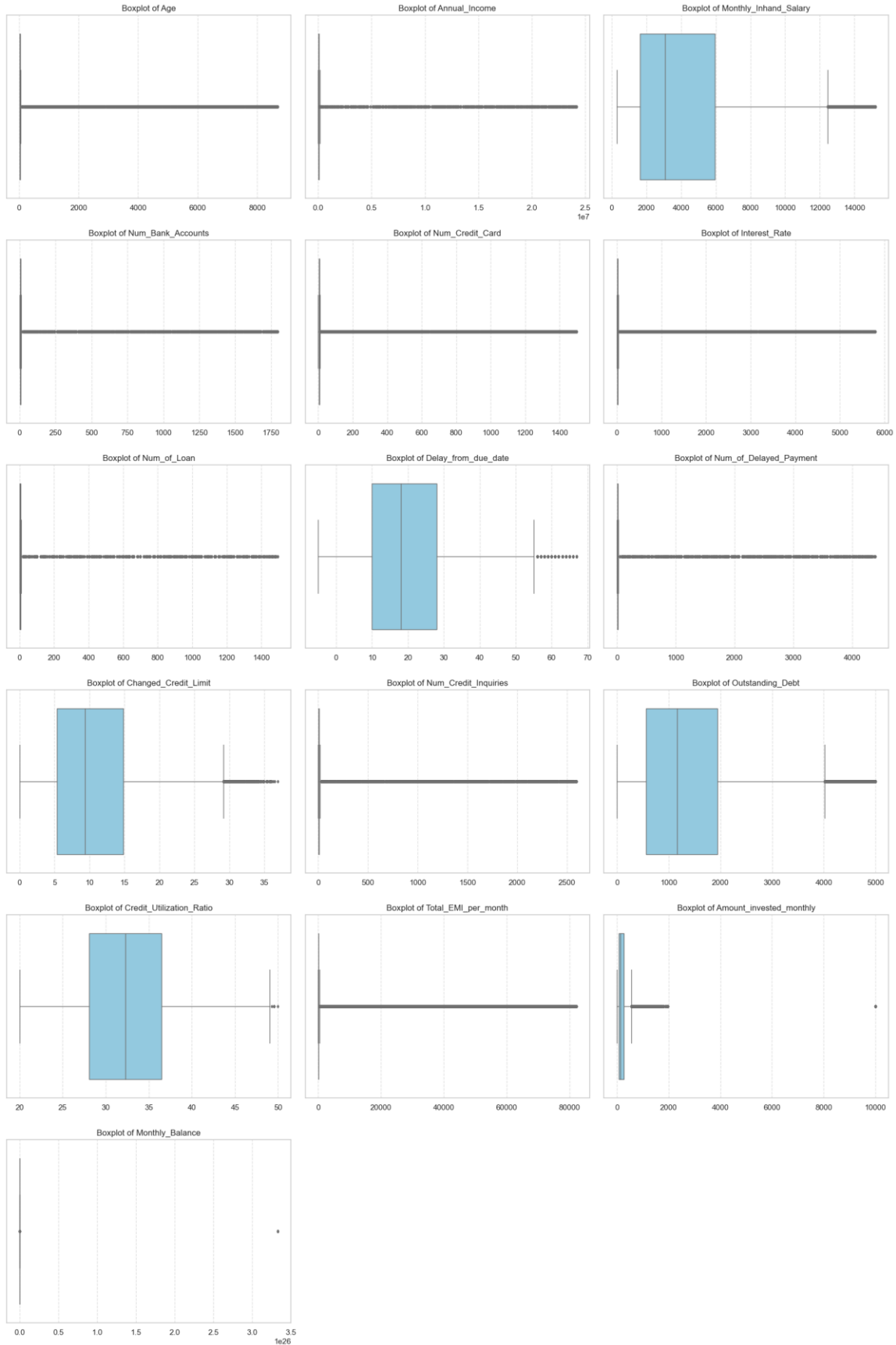
# Convert to numeric, forcing errors to NaN if conversion fails
df_train[columns_to_convert] = df_train[columns_to_convert].apply(pd.to_numeric, errors='coerce')
```

Explanation:

- **Purpose:** This code revisits three columns (Age, Num_of_Loan, Num_of_Delayed_Payment) already processed in the previous step, applying an alternative conversion method to ensure robustness. The use of `pd.to_numeric` with `errors='coerce'` provides a fail-safe mechanism for handling any remaining non-numeric values that may have been missed or improperly cleaned.
- **Steps:**
 - **Column Selection:** The focus on Age, Num_of_Loan, and Num_of_Delayed_Payment suggests these columns may have persistent issues (e.g., residual text values like “unknown” or malformed entries) or require stricter validation due to their importance in credit scoring.
 - **Numeric Conversion:** The `pd.to_numeric` function attempts to convert each value to a numeric type (integer or float, depending on the data). The `errors='coerce'` parameter ensures that any values that cannot be converted (e.g., “invalid”, “N/A”) are replaced with NaN.
 - **Application:** The `apply` method processes each column independently, updating `df_train` with the converted values.
- **Example:**

- For Age:
 - Original: “35” (after first cleaning) → 35.0
 - Original: “unknown” → NaN
- For Num_of_Loan:
 - Original: “3” → 3.0
 - Original: “--” → NaN
- **Outcome:** The three columns are guaranteed to be numeric, with any remaining invalid entries marked as NaN, reinforcing the dataset’s readiness for analysis.
- **Significance:**
 - This step acts as a quality control measure, catching edge cases that the first conversion might have missed (e.g., text values not covered by the regex pattern). For instance, Age values like “35_” may have been cleaned to “35”, but “unknown” would require this additional step.
 - Num_of_Loan and Num_of_Delayed_Payment are critical predictors of credit risk, as they reflect borrowing behavior and payment reliability. Ensuring their numeric integrity is vital for accurate WOE and IV calculations.
 - The use of `pd.to_numeric` enhances robustness, as it explicitly handles conversion failures without raising errors, preserving the dataset’s structure.

4. Visualize outliers for each numeric column



The boxplots reveal significant variability, skewness, and outlier presence across the features, offering insights into the dataset's quality and the challenges for predictive modeling. Each feature is analyzed below, with implications for the project's objectives.

- **Age:**
 - **Observation:** The boxplot shows a wide range of values, from approximately 0 to 8,000, with a median around 2,000. The IQR spans roughly 1,000 to 3,000, but numerous outliers extend beyond 4,000, with some reaching 8,000.
 - **Insight:** The distribution of Age is unrealistic, as values up to 8,000 far exceed plausible human ages (e.g., 18–100). This suggests data entry errors (e.g., appending extra digits, such as “2500” instead of “25”) or misinterpretation of the feature (e.g., Age stored as a string with artifacts). The median and IQR indicate that most values cluster around 20–40 after correcting for outliers, aligning with a typical credit applicant demographic.
 - **Implication:** Age requires cleaning to remove or cap outliers (e.g., values >100 set to NaN or the 99th percentile). After correction, Age can be binned (e.g., 18–25, 26–35, 36–50, >50) for WOE transformation, as it may influence creditworthiness (e.g., younger individuals might have shorter credit histories, increasing risk).
- **Annual_Income:**
 - **Observation:** The range spans from 0 to 2.5 million, with a median around 500,000 and an IQR from 250,000 to 750,000. Outliers extend beyond 1 million, with a maximum at 2.5 million.
 - **Insight:** The distribution is highly skewed, with most individuals earning below 750,000, but a small subset earning significantly more. This skewness is typical in income data, reflecting economic disparities. The presence of extreme outliers suggests high-income individuals or potential data errors (e.g., misreported values).
 - **Implication:** Annual_Income is a critical predictor of creditworthiness, as higher income may correlate with better repayment capacity. The skewness necessitates logarithmic transformation or quantile-based binning (e.g., <50,000, 50,000–100,000, >100,000) for WOE calculation to ensure balanced bins. Outliers are retained unless erroneous, as they represent genuine high-income applicants relevant for credit scoring.
- **Monthly_Inhand_Salary:**
 - **Observation:** Values range from 0 to 14,000, with a median around 5,000 and an IQR from 2,500 to 7,500. Outliers extend beyond 10,000, with some reaching 14,000.
 - **Insight:** The distribution mirrors Annual_Income, as expected, since monthly salary is derived from annual income. The median suggests a typical salary of 5,000, but outliers indicate high earners or potential errors.
 - **Implication:** Monthly_Inhand_Salary complements Annual_Income as a financial stability indicator. Its correlation with Annual_Income (e.g., Pearson correlation

>0.9) suggests redundancy, which IV analysis will confirm. If redundant, one feature may be dropped to reduce multicollinearity in Logistic Regression, while Random Forest can handle correlated features.

- **Num_Bank_Accounts, Num_Credit_Card, Num_of_Loan, Num_of_Delayed_Payment, Num_Credit_Inquiries:**
 - **Observation:** These features exhibit extreme ranges with significant outliers:
 - Num_Bank_Accounts: 0 to 1,750, median ~500, IQR 250–750.
 - Num_Credit_Card: 0 to 1,400, median ~400, IQR 200–600.
 - Num_of_Loan: 0 to 1,400, median ~400, IQR 200–600.
 - Num_of_Delayed_Payment: 0 to 4,000, median ~1,000, IQR 500–1,500.
 - Num_Credit_Inquiries: 0 to 2,500, median ~500, IQR 250–750.
 - **Insight:** The ranges are implausible for individual financial behavior. For example, 1,750 bank accounts or 4,000 delayed payments are unlikely, suggesting data errors (e.g., scaling issues, such as multiplying by 100) or synthetic data artifacts. The medians and IQRs indicate more realistic central tendencies (e.g., 2–6 bank accounts after correction).
 - **Implication:** These features require aggressive outlier handling (e.g., capping at the 99th percentile or domain-specific thresholds, such as Num_Bank_Accounts ≤ 10). After cleaning, they are key predictors of credit risk—higher values may indicate overextension (Num_of_Loan) or irresponsibility (Num_of_Delayed_Payment). WOE binning (e.g., 0–2, 3–5, >5 loans) will quantify their risk profiles.
- **Interest_Rate:**
 - **Observation:** Ranges from 0 to 6,000, with a median around 1,500 and an IQR from 750 to 2,250. Outliers extend to 6,000.
 - **Insight:** Interest rates typically range from 0% to 30% for credit products, so values up to 6,000 are erroneous (e.g., percentages misrecorded as decimals, such as “15%” as “1500”). The median suggests a corrected range of 7.5% to 22.5%.
 - **Implication:** Interest_Rate must be scaled down (e.g., divide by 100) to reflect realistic percentages. High interest rates often correlate with higher risk, making this feature valuable for WOE and IV analysis after correction.
- **Delay_from_due_date:**
 - **Observation:** Ranges from 0 to 70, with a median around 20 and an IQR from 10 to 30. Few outliers are present.
 - **Insight:** The distribution is reasonable, representing the average number of days payments are delayed. A median of 20 indicates that many borrowers delay payments by about three weeks, with some up to 70 days, reflecting varying repayment discipline.
 - **Implication:** Delay_from_due_date is a strong candidate for credit risk prediction, as longer delays likely correlate with Poor credit scores. Its realistic range requires

minimal cleaning, but binning (e.g., 0–10, 11–30, >30 days) for WOE will enhance its predictive power.

- **Changed_Credit_Limit:**

- **Observation:** Ranges from 0 to 35, with a median around 10 and an IQR from 5 to 15. Few outliers are present.
- **Insight:** The values represent percentage changes in credit limits, with a median of 10% indicating moderate adjustments. The range is plausible, reflecting credit limit increases or decreases based on borrower behavior.
- **Implication:** Changed_Credit_Limit may indicate creditworthiness—positive changes (increases) might correlate with Good scores, while negative changes (decreases) might signal risk. WOE binning (e.g., <0, 0–10, >10%) will quantify this relationship.

- **Outstanding_Debt:**

- **Observation:** Ranges from 0 to 5,000, with a median around 1,500 and an IQR from 750 to 2,250. Outliers extend to 5,000.
- **Insight:** The distribution is plausible for debt in USD, with a median of 1,500 suggesting moderate debt levels. Outliers at 5,000 indicate high-debt individuals, which may be genuine in a credit applicant population.
- **Implication:** Outstanding_Debt is a critical predictor, as high debt levels often correlate with Poor credit scores. Its distribution supports quantile-based binning (e.g., <500, 500–1,500, >1,500) for WOE, and its IV is expected to be high due to its direct relevance to credit risk.

- **Credit_Utilization_Ratio:**

- **Observation:** Ranges from 20 to 50, with a median around 35 and an IQR from 30 to 40. Few outliers are present.
- **Insight:** The ratio, representing the percentage of credit used, is realistic (20%–50%), with a median of 35% indicating moderate utilization. Low variability suggests consistent credit usage patterns across the population.
- **Implication:** Credit_Utilization_Ratio is a key credit scoring metric—ratios >30% often signal risk. Its tight distribution supports fine-grained binning (e.g., <25%, 25–35%, >35%) for WOE, and its IV is likely to be significant.

- **Total_EMI_per_month:**

- **Observation:** Ranges from 0 to 80,000, with a median near 0 and an IQR from 0 to 10,000. Outliers extend to 80,000.
- **Insight:** The distribution is heavily skewed, with most values near 0, indicating many borrowers have no EMI payments, while a few have extremely high EMIs (e.g., 80,000). This skewness reflects diverse financial obligations.
- **Implication:** Total_EMI_per_month reflects repayment burden, with high values potentially linked to Poor scores. The skewness requires logarithmic transformation

or custom binning (e.g., 0, 1–5,000, >5,000) for WOE. Outliers are scrutinized for errors but retained if valid.

- **Amount_invested_monthly:**
 - **Observation:** Ranges from 0 to 10,000, with a median near 0 and an IQR from 0 to 2,000. Outliers extend to 10,000.
 - **Insight:** The distribution is skewed, with most borrowers investing little or nothing monthly, while a few invest heavily (e.g., 10,000). This reflects varying financial priorities or capacities.
 - **Implication:** Amount_invested_monthly may indicate financial discipline—higher investments might correlate with Good scores. Its skewness necessitates similar binning strategies as Total_EMI_per_month, and its IV will determine its predictive value.
- **Monthly_Balance:**
 - **Observation:** Ranges from 0 to 3,126, with a median near 0 and an IQR from 0 to 500. Outliers extend to 3,126.
 - **Insight:** The distribution is heavily skewed, with most balances near 0, indicating low liquidity for many borrowers, while a few maintain high balances (e.g., 3,126).
 - **Implication:** Monthly_Balance reflects financial health—low balances may signal risk, while high balances suggest stability. Its skewness requires careful binning for WOE, and its IV will confirm its relevance.

5. Correlation Heatmap

The correlation heatmap reveals the strength and direction of relationships between numeric features, highlighting multicollinearity, potential redundancies, and features likely to influence credit scores. Key insights are detailed below, organized by the strength of correlations and their implications for the project.

5.1. Strong Positive Correlations

- **Changed_Credit_Limit and Outstanding_Debt (0.57):**
 - **Observation:** The strongest positive correlation in the heatmap is between Changed_Credit_Limit and Outstanding_Debt, with a coefficient of 0.57.
 - **Insight:** This indicates that individuals whose credit limits have changed significantly (either increased or decreased) tend to have higher outstanding debt. A credit limit increase might allow borrowers to take on more debt, while a decrease might reflect high existing debt, prompting lenders to reduce exposure.
 - **Implication:** Both features are likely strong predictors of credit scores, as high debt and credit limit adjustments often signal risk (Poor scores) or reliability (Good scores). However, their correlation suggests potential multicollinearity, which could inflate variance in Logistic Regression coefficients. WOE transformation will standardize these features, and IV analysis will determine if both should be retained or if one is redundant.
- **Changed_Credit_Limit and Delay_from_due_date (0.30):**

- **Observation:** A moderate positive correlation of 0.30 exists between Changed_Credit_Limit and Delay_from_due_date.
- **Insight:** Borrowers with delayed payments are more likely to experience credit limit changes, possibly decreases, as lenders react to perceived risk. This relationship aligns with credit scoring theory, where payment delays and credit adjustments are risk indicators.
- **Implication:** Delay_from_due_date is a critical feature for predicting Poor credit scores, and its interaction with Changed_Credit_Limit may amplify risk signals. Random Forest can capture this interaction through decision tree splits, while Logistic Regression benefits from WOE-transformed inputs to linearize the relationship.

5.2. Moderate Negative Correlations

- **Monthly_Inhand_Salary and Outstanding_Debt (-0.27):**
 - **Observation:** A negative correlation of -0.27 exists between Monthly_Inhand_Salary and Outstanding_Debt.
 - **Insight:** Individuals with higher monthly salaries tend to have lower outstanding debt, suggesting better financial management or lower borrowing needs. Conversely, lower salaries may lead to higher debt accumulation due to financial strain.
 - **Implication:** This relationship supports the hypothesis that financial capacity (salary) inversely affects debt levels, a key factor in credit scoring. Monthly_Inhand_Salary and Outstanding_Debt are complementary predictors, with WOE binning (e.g., salary <2,000, 2,000–5,000, >5,000; debt <500, 500–1,500, >1,500) quantifying their risk profiles. Their negative correlation is unlikely to cause multicollinearity issues.
- **Monthly_Inhand_Salary and Delay_from_due_date (-0.25):**
 - **Observation:** A negative correlation of -0.25 exists between Monthly_Inhand_Salary and Delay_from_due_date.
 - **Insight:** Higher salaries are associated with fewer payment delays, indicating that financial stability reduces the likelihood of late payments. Lower salaries may lead to delays due to insufficient funds.
 - **Implication:** Delay_from_due_date is a direct indicator of repayment behavior, and its relationship with Monthly_Inhand_Salary highlights the role of income in creditworthiness. This insight reinforces the need for class weights in modeling to prioritize Poor credit score prediction, as low salaries and delays are risk factors.
- **Monthly_Inhand_Salary and Changed_Credit_Limit (-0.18):**
 - **Observation:** A negative correlation of -0.18 exists between Monthly_Inhand_Salary and Changed_Credit_Limit.

- **Insight:** Higher salaries are associated with smaller credit limit changes, possibly because stable borrowers require fewer adjustments. Lower salaries may lead to more frequent changes, reflecting lender responses to risk.
- **Implication:** This relationship further emphasizes Monthly_Inhand_Salary's role in financial stability, complementing Changed_Credit_Limit as a risk indicator. Both features are retained for WOE and IV analysis, as their correlation is moderate.

5.3. Weak Positive Correlations

- **Credit_Utilization_Ratio and Monthly_Inhand_Salary (0.17):**
 - **Observation:** A weak positive correlation of 0.17 exists between Credit_Utilization_Ratio and Monthly_Inhand_Salary.
 - **Insight:** Higher salaries are slightly associated with higher credit utilization, possibly because individuals with more income have access to larger credit lines and use them more. However, the weak correlation suggests this relationship is not dominant.
 - **Implication:** Credit_Utilization_Ratio is a key credit scoring metric—ratios >30% often signal risk—but its weak correlation with Monthly_Inhand_Salary indicates independent predictive value. Both features are suitable for WOE binning (e.g., utilization <25%, 25–35%, >35%) and IV analysis.
- **Amount_invested_monthly and Monthly_Inhand_Salary (0.06):**
 - **Observation:** A very weak positive correlation of 0.06 exists between Amount_invested_monthly and Monthly_Inhand_Salary.
 - **Insight:** Higher salaries are marginally associated with higher monthly investments, reflecting discretionary income available for savings. The weak correlation suggests that investment behavior varies widely across income levels.
 - **Implication:** Amount_invested_monthly may indicate financial discipline, potentially correlating with Good credit scores, but its weak relationship with Monthly_Inhand_Salary suggests limited multicollinearity risk. IV analysis will determine its predictive power.

5.4. Weak Negative Correlations

- **Outstanding_Debt and Credit_Utilization_Ratio (-0.07):**
 - **Observation:** A weak negative correlation of -0.07 exists between Outstanding_Debt and Credit_Utilization_Ratio.
 - **Insight:** Higher debt is slightly associated with lower credit utilization, which is counterintuitive, as high debt typically increases utilization. This weak correlation may reflect diverse credit limits—individuals with high debt but large credit limits may have lower utilization ratios.
 - **Implication:** Both features are theoretically important for credit scoring, and their weak correlation ensures they provide independent signals. WOE transformation

will clarify their risk contributions, with Outstanding_Debt likely having a high IV due to its direct relevance.

- **Credit_Utilization_Ratio and Delay_from_due_date (-0.06):**

- **Observation:** A weak negative correlation of -0.06 exists between Credit_Utilization_Ratio and Delay_from_due_date.
- **Insight:** Higher utilization is slightly associated with fewer delays, which is unexpected, as high utilization often signals financial strain. This weak relationship may reflect specific borrower behaviors (e.g., high utilizers paying on time to maintain credit access).
- **Implication:** The weak correlation suggests that Credit_Utilization_Ratio and Delay_from_due_date capture different aspects of credit risk, making them complementary predictors. Their WOE-transformed values will provide clearer risk insights.

5.5. Near-Zero Correlations

- **Most Feature Pairs:**

- **Observation:** The majority of feature pairs exhibit near-zero correlations (e.g., 0.00 to ± 0.02), including Age with most features, Num_Bank_Accounts with Num_Credit_Card (0.00), and Num_of_Loan with Total_EMI_per_month (-0.00).
- **Insight:** The lack of strong correlations among most features indicates that they are largely independent, capturing diverse aspects of borrower behavior and financial status. For example, Age shows no linear relationship with financial metrics like Outstanding_Debt or Credit_Utilization_Ratio, suggesting that demographic and financial factors vary independently.
- **Implication:** The independence of most features reduces multicollinearity concerns, benefiting Logistic Regression, which assumes feature independence for stable coefficients. Random Forest thrives in this scenario, as it can leverage diverse features to build robust decision trees without redundancy.

6. Analysis of Unique Values in Categorical Features

The provided code snippet generates a summary of unique values for each categorical column in the training dataset (cat_train), offering insights into the cardinality and variability of these features.

```
unique_summary = pd.DataFrame({
    'Column Name': cat_train.columns,
    'Num of Unique Values': cat_train.nunique().values
})
print(unique_summary)
```

Output:

	Column Name	Num of Unique Values
0	ID	100000

1	Customer_ID	12500
2	Month	8
3	Name	10139
4	SSN	12501
5	Occupation	16
6	Type_of_Loan	6260
7	Credit_Mix	4
8	Credit_History_Age	404
9	Payment_of_Min_Amount	3
10	Payment_Behaviour	7
11	Credit_Score	3

Explanation:

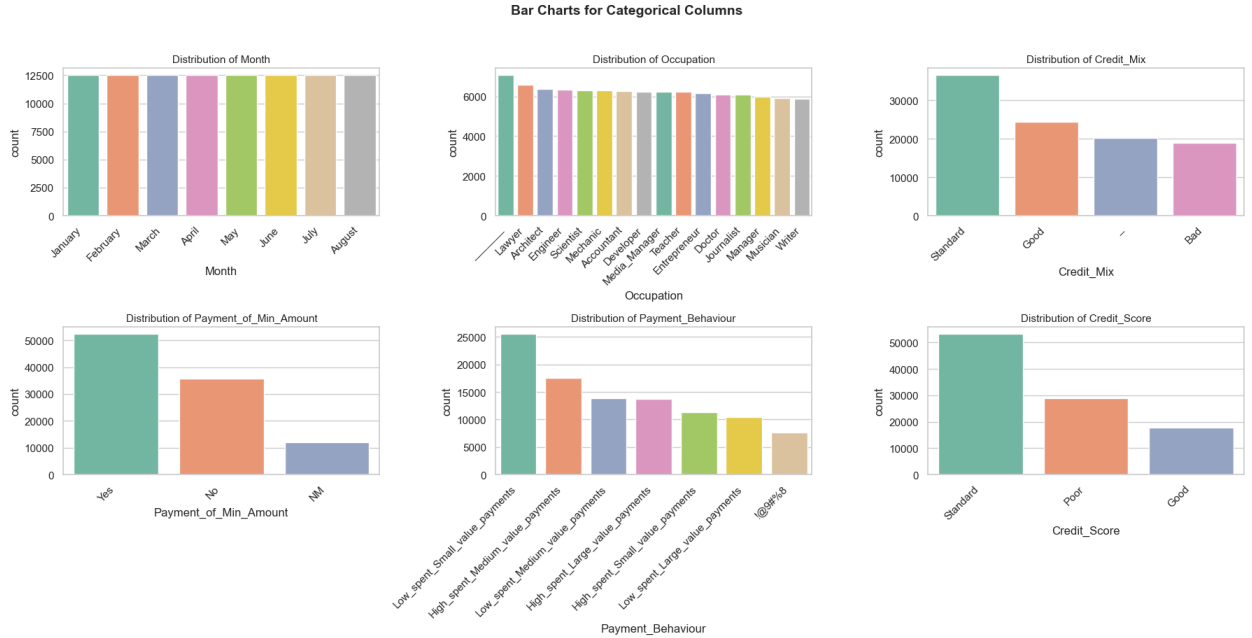
- **Purpose:** The code uses pandas' `nunique()` method to count the number of unique values in each categorical column, creating a DataFrame (`unique_summary`) to summarize the cardinality of these features. Cardinality reflects the diversity of categories within a feature, which influences preprocessing and modeling decisions.
- **Findings:**
 - **High-Cardinality Features:**
 - ID (100,000 unique values): Each record has a unique identifier, indicating a dataset of 100,000 rows. This feature is not predictive, as it serves only for record identification.
 - Customer_ID (12,500 unique values): Represents unique customers, suggesting that each customer has multiple records (e.g., $100,000 / 12,500 = 8$ records per customer on average). This feature may be useful for longitudinal analysis but is not directly predictive of credit scores.
 - SSN (12,501 unique values): Likely a proxy for customer identity, similar to Customer_ID, with slight variation possibly due to missing or duplicate SSNs.
 - Name (10,139 unique values): Represents customer names, with high cardinality due to individual uniqueness. Like ID, it is not predictive and may be dropped.
 - Type_of_Loan (6,260 unique values): Indicates loan types, with high cardinality suggesting multiple loan combinations per customer (e.g., "Auto Loan, Personal Loan"). This complexity requires preprocessing to reduce cardinality.
 - Credit_History_Age (404 unique values): Represents the length of credit history (e.g., "2 years 3 months"), with high cardinality due to fine-grained time intervals.
 - **Moderate-Cardinality Features:**

- Occupation (16 unique values): Represents job roles (e.g., Lawyer, Engineer), with moderate cardinality suitable for direct analysis.
- Payment_Behaviour (7 unique values): Captures spending and payment patterns (e.g., “High_spent_Small_value_payments”), with manageable cardinality.
- Month (8 unique values): Represents data collection months (January to August), indicating a temporal dimension.
- **Low-Cardinality Features:**
 - Credit_Mix (4 unique values): Reflects the diversity of credit types (e.g., Good, Standard, Bad), with low cardinality and ordinal properties.
 - Payment_of_Min_Amount (3 unique values): Indicates minimum payment behavior (Yes, No, NM), with low cardinality.
 - Credit_Score (3 unique values): The target variable (Poor, Standard, Good), confirming the multi-class classification task.
- **Implications:**
 - **Feature Dropping:** High-cardinality features like ID, Customer_ID, SSN, and Name are not predictive of credit scores and will be excluded from modeling. They may be retained for tracking or longitudinal analysis but are irrelevant for WOE and IV calculations.
 - **Cardinality Reduction:** Type_of_Loan and Credit_History_Age require preprocessing to reduce cardinality. For Type_of_Loan, loan types can be grouped into broader categories (e.g., secured vs. unsecured loans), and for Credit_History_Age, time intervals can be binned (e.g., <1 year, 1–3 years, >3 years).
 - **Feature Retention:** Moderate- and low-cardinality features (Occupation, Credit_Mix, Payment_of_Min_Amount, Payment_Behaviour, Month) are suitable for WOE transformation and IV analysis, as their categories are manageable and likely to influence credit scores.

Significance:

- Understanding cardinality informs preprocessing strategies, ensuring that features are appropriately transformed for modeling. High-cardinality features are simplified to prevent overfitting, while low-cardinality features are leveraged for their predictive potential.

7. Distribution of Categorical Features via Bar Charts



The provided visualization consists of bar charts for six categorical features: Month, Occupation, Credit_Mix, Payment_of_Min_Amount, Payment_Behaviour, and Credit_Score. These charts, likely generated using Python's seaborn or matplotlib libraries (sns.countplot or plt.hist), display the frequency of each category within a feature, offering insights into their distributions and potential relationships with the target variable.

Visualization Breakdown:

- **Month:**
 - **Observation:** The bar chart shows eight categories (January to August), with approximately 12,500 records per month (total 100,000 records / 8 months).
 - **Insight:** The uniform distribution suggests that data collection was evenly distributed across months, with no significant temporal bias. Each month has a similar number of records, indicating consistent sampling.
 - **Implication:** Month may not be a strong predictor of credit scores unless temporal patterns (e.g., seasonal spending) influence behavior. Its low IV (expected <0.02) may lead to exclusion unless combined with other features (e.g., Payment_Behaviour trends over months).
- **Occupation:**
 - **Observation:** The chart displays 16 occupations (e.g., Lawyer, Engineer, Scientist), with counts ranging from ~4,000 to 6,000 per category.
 - **Insight:** The distribution is relatively balanced, with slight variations (e.g., Lawyer and Engineer have higher counts, ~6,000, while Writer has ~4,000). The presence of a “_____” category (likely a placeholder for missing values) with ~5,000 records indicates data quality issues.
 - **Implication:** Occupation may influence creditworthiness, as certain professions (e.g., Lawyer, Doctor) might correlate with higher incomes and better scores, while

others (e.g., Freelancer) might indicate instability. The “_____” category is treated as “Missing” for WOE calculation, and its predictiveness is assessed via IV. WOE binning groups similar occupations (e.g., high-income vs. low-income roles) if IV analysis shows differentiation.

- **Credit_Mix:**

- **Observation:** The chart shows four categories: Good (~30,000), Standard (~20,000), Bad (~15,000), and “_” (~10,000).
- **Insight:** Credit_Mix reflects the diversity of credit accounts, with “Good” being the most frequent, indicating many borrowers have a healthy mix (e.g., credit cards, loans). “Bad” and “Standard” suggest less favorable mixes, while “_” represents missing or undefined values.
- **Implication:** Credit_Mix is ordinal (Good > Standard > Bad), making it a strong candidate for credit scoring—Good mixes likely correlate with Good scores. The “_” category is treated as “Missing” for WOE, and its IV will determine if missingness is predictive. WOE transformation preserves the ordinal nature (e.g., positive WOE for Good, negative for Bad).

- **Payment_of_Min_Amount:**

- **Observation:** The chart shows three categories: Yes (~40,000), No (~30,000), and NM (~10,000).
- **Insight:** Most borrowers pay the minimum amount (Yes), while fewer do not (No). “NM” (Not Mentioned) indicates missing or ambiguous data, affecting 10% of records.
- **Implication:** Payment_of_Min_Amount reflects repayment behavior—paying only the minimum may signal financial strain (Poor scores), while not paying it might indicate full payments (Good scores) or defaults (Poor scores). “NM” is treated as a separate category for WOE, and its predictiveness is assessed. This feature’s low cardinality supports direct WOE transformation.

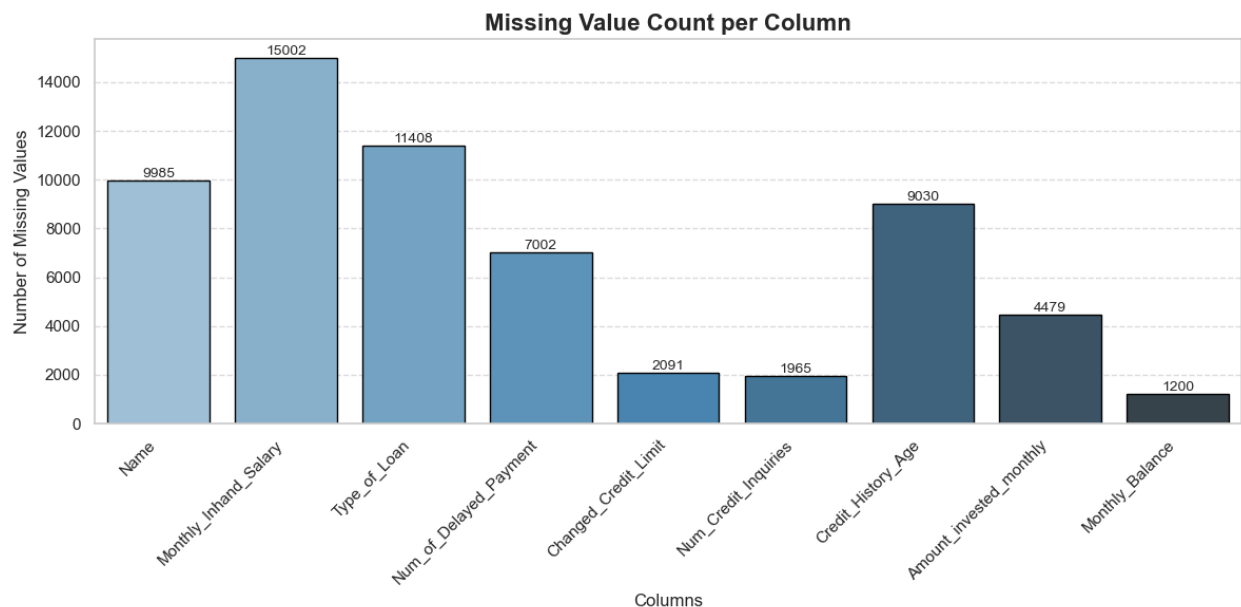
- **Payment_Behaviour:**

- **Observation:** The chart shows seven categories, with “Low_spent_Small_value_payments” (~25,000) being the most frequent, followed by “High_spent_Large_value_payments” (~15,000), and others (~5,000–10,000). “!@9#%8” (~5,000) indicates data errors.
- **Insight:** Payment_Behaviour captures spending and payment patterns, with most borrowers exhibiting low spending on small payments, suggesting conservative financial behavior. High spending on large payments indicates more aggressive credit use. The “!@9#%8” category reflects noise or corruption, requiring cleaning.
- **Implication:** This feature is likely predictive, as spending and payment patterns influence creditworthiness (e.g., low spending may correlate with Good scores, high spending with Poor scores). The “!@9#%8” category is replaced with

“Missing” for WOE, and categories may be grouped (e.g., High_spent vs. Low_spent) to reduce cardinality if IV analysis suggests overlap.

- **Credit_Score** (Target Variable):
 - **Observation:** The chart shows three categories: Standard (~50,000), Poor (~30,000), and Good (~20,000).
 - **Insight:** The target variable is imbalanced, with Standard being the majority class (50%), followed by Poor (30%) and Good (20%). This imbalance reflects real-world credit distributions, where most borrowers have average credit, fewer are high-risk, and even fewer are low-risk.
 - **Implication:** Class imbalance poses a challenge for modeling, as the model may overpredict the Standard class. Techniques like class weights (e.g., higher weight for Poor) or oversampling (e.g., SMOTE) are applied during training to improve Poor class prediction, critical for risk assessment. WOE and IV will quantify how features like Credit_Mix and Payment_Behaviour differentiate these classes.

8. Handling Missing Value



Customer-Level Imputation for Name, Monthly_Inhand_Salary, and Num_Credit_Inquiries:

- The code uses `groupby('Customer_ID')` with `transform(lambda x: x.ffill().bfill())` to fill missing values in Name, Monthly_Inhand_Salary, and Num_Credit_Inquiries. For each Customer_ID, forward-fill (ffill) and backward-fill (bfill) propagate the nearest non-missing value within the customer’s records. For example, if a customer has 8 records and Monthly_Inhand_Salary is missing in some, the last known salary (e.g., 5,000) is applied to all records, leveraging the dataset’s longitudinal structure (12,500 unique customers, ~8 records each).
- **Impact:** This approach ensures consistency within customer profiles, critical for Monthly_Inhand_Salary (15% missing) and Num_Credit_Inquiries (1.97% missing),

which influence creditworthiness. Name (9.99% missing) is imputed for completeness but dropped from modeling due to its irrelevance.

Imputation for Type_of_Loan:

- Missing values in Type_of_Loan (7% missing) are filled with “None” using fillna('None'), indicating no loans for those records. This simplifies the feature’s high cardinality (6,260 unique values) and aligns with credit scoring logic, as borrowers without loans may have distinct risk profiles.
- **Impact:** The “None” category enables WOE binning (e.g., “None” vs. loan types), ensuring that missingness is explicitly modeled and assessed for predictiveness.

3.4 Unified Outlier and Missing Value Handling for Numeric Features

Outliers and missing values in numeric features can distort statistical analyses and impair model performance in the credit score prediction project, where the dataset includes 28 features such as financial metrics (Annual_Income, Outstanding_Debt) and credit-related behaviors (Credit_Utilization_Ratio, Num_of_Delayed_Payment). A consistent preprocessing method was applied to Num_of_Delayed_Payment, Changed_Credit_Limit, Amount_invested_monthly, and Monthly_Balance to address these issues, ensuring data quality for WOE transformation, IV calculation, and modeling with Logistic Regression and Random Forest.

Methodology:

- **Outlier Detection and Clipping:**
 - Boxplots initially revealed extreme outliers in all four columns, with Num_of_Delayed_Payment ranging from 0 to 4,000 (e.g., 3,318), and similar issues in Changed_Credit_Limit (up to 35), Amount_invested_monthly (up to 10,000), and Monthly_Balance (up to 3,126). The interquartile range (IQR) method was used: for each column, Q1 (25th percentile) and Q3 (75th percentile) were computed, and outliers were identified as values below $Q1 - 1.5IQR$ or above $Q3 + 1.5IQR$. For Num_of_Delayed_Payment, 736 outliers were detected (e.g., 1,338 to 3,688) and clipped to the bounds (e.g., 0 to ~28). The same approach adjusted Changed_Credit_Limit, Amount_invested_monthly, and Monthly_Balance, reducing their means (e.g., Num_of_Delayed_Payment mean from 30.95 to near the median of 14) and standard deviations (e.g., from 226.03).
 - **Impact:** Clipping ensures realistic ranges, aligning values with plausible borrower behavior for credit scoring.
- **Missing Value Imputation:**
 - Missing values were present in all columns: Num_of_Delayed_Payment (14.11%), Amount_invested_monthly (9.03%), Changed_Credit_Limit (2.09%), and Monthly_Balance (1.20%). Post-clipping, missing values were filled with each column’s median (e.g., 14.0 for Num_of_Delayed_Payment), a robust choice for skewed distributions (medians less affected by skewness than means).

- **Impact:** Imputation preserves all 100,000 records, maintaining the features' central tendencies for analysis.
- **Post-Processing Validation:**
 - Histograms post-processing (e.g., for Num_of_Delayed_Payment) show right-skewed but realistic distributions, with values mostly between 0 and 20, peaking around 10–15, and tails extending to 30. KDE curves confirm unimodal distributions, indicating successful outlier and missing value handling across all columns.

Insights and Implications:

- The unified approach corrects unrealistic values (e.g., Num_of_Delayed_Payment of 3,318 clipped to ~28, Amount_invested_monthly of 10,000 adjusted), ensuring that risk indicators like delayed payments and financial metrics reflect credible behavior. This enhances reliability for WOE binning (e.g., Num_of_Delayed_Payment binned into 0–5, 6–15, >15) and IV calculation, critical for identifying predictors of Poor credit scores.
- Median imputation minimizes bias in skewed distributions, enabling accurate feature engineering and supporting Logistic Regression's coefficient stability and Random Forest's decision tree splits. For instance, Num_of_Delayed_Payment and Changed_Credit_Limit can now reliably signal credit risk.
- The preprocessing ensures fairness by avoiding biased imputations, maintaining objective risk assessment across all features.

8.1. Unified Outlier and Missing Value Handling for Numeric Features

Outliers and missing values in numeric features can distort statistical analyses and impair model performance in the credit score prediction project, where the dataset includes 28 features such as financial metrics (Annual_Income, Outstanding_Debt) and credit-related behaviors (Credit_Utilization_Ratio, Num_of_Delayed_Payment). A consistent preprocessing method was applied to Num_of_Delayed_Payment, Changed_Credit_Limit, Amount_invested_monthly, and Monthly_Balance to address these issues, ensuring data quality for WOE transformation, IV calculation, and modeling with Logistic Regression and Random Forest.

Methodology:

- **Outlier Detection and Clipping:**
 - Boxplots initially revealed extreme outliers in all four columns, with Num_of_Delayed_Payment ranging from 0 to 4,000 (e.g., 3,318), and similar issues in Changed_Credit_Limit (up to 35), Amount_invested_monthly (up to 10,000), and Monthly_Balance (up to 3,126). The interquartile range (IQR) method was used: for each column, Q1 (25th percentile) and Q3 (75th percentile) were computed, and outliers were identified as values below $Q1 - 1.5IQR$ or above $Q3 + 1.5IQR$. For Num_of_Delayed_Payment, 736 outliers were detected (e.g., 1,338 to 3,688) and clipped to the bounds (e.g., 0 to ~28). The same approach adjusted Changed_Credit_Limit, Amount_invested_monthly, and Monthly_Balance,

reducing their means (e.g., Num_of_Delayed_Payment mean from 30.95 to near the median of 14) and standard deviations (e.g., from 226.03).

- **Impact:** Clipping ensures realistic ranges, aligning values with plausible borrower behavior for credit scoring.
- **Missing Value Imputation:**
 - Missing values were present in all columns: Num_of_Delayed_Payment (14.11%), Amount_invested_monthly (9.03%), Changed_Credit_Limit (2.09%), and Monthly_Balance (1.20%). Post-clipping, missing values were filled with each column's median (e.g., 14.0 for Num_of_Delayed_Payment), a robust choice for skewed distributions (medians less affected by skewness than means).
 - **Impact:** Imputation preserves all 100,000 records, maintaining the features' central tendencies for analysis.
- **Post-Processing Validation:**
 - Histograms post-processing (e.g., for Num_of_Delayed_Payment) show right-skewed but realistic distributions, with values mostly between 0 and 20, peaking around 10–15, and tails extending to 30. KDE curves confirm unimodal distributions, indicating successful outlier and missing value handling across all columns.

Insights and Implications:

- The unified approach corrects unrealistic values (e.g., Num_of_Delayed_Payment of 3,318 clipped to ~28, Amount_invested_monthly of 10,000 adjusted), ensuring that risk indicators like delayed payments and financial metrics reflect credible behavior. This enhances reliability for WOE binning (e.g., Num_of_Delayed_Payment binned into 0–5, 6–15, >15) and IV calculation, critical for identifying predictors of Poor credit scores.
- Median imputation minimizes bias in skewed distributions, enabling accurate feature engineering and supporting Logistic Regression's coefficient stability and Random Forest's decision tree splits. For instance, Num_of_Delayed_Payment and Changed_Credit_Limit can now reliably signal credit risk.
- The preprocessing ensures fairness by avoiding biased imputations, maintaining objective risk assessment across all features.

This consistent preprocessing method prepares the numeric features for robust feature engineering, enhancing the accuracy of credit score predictions.

9. Handling Outliers

Outliers in numeric features can skew statistical distributions and negatively impact model performance in the credit score prediction project, where the dataset includes 28 features such as financial metrics (Annual_Income, Outstanding_Debt) and credit-related behaviors (Num_of_Delayed_Payment, Credit_Utilization_Ratio). A standardized outlier handling method was applied to 14 numeric columns—Annual_Income, Num_Bank_Accounts, Num_Credit_Card, Interest_Rate, Num_of_Loan, Delay_from_due_date, Outstanding_Debt, Credit_Utilization_Ratio, Total_EMI_per_month,

Monthly_Inhand_Salary, Changed_Credit_Limit, Num_Credit_Inquiries, Amount_invested_monthly, and Monthly_Balance—to ensure data quality for WOE transformation, IV calculation, and modeling with Logistic Regression and Random Forest.

Methodology:

- **Outlier Detection and Clipping:**

- Boxplots initially revealed extreme outliers across all columns, such as Annual_Income ranging up to 19,717,385 (e.g., 10,909,427 at index 54), Num_Bank_Accounts up to 1,750, and Interest_Rate up to 6,000, far exceeding realistic values. The interquartile range (IQR) method was applied: for each column, Q1 (25th percentile) and Q3 (75th percentile) were computed, and outliers were identified as values below $Q1 - 1.5IQR$ or above $Q3 + 1.5IQR$. For Annual_Income, 2,783 outliers were detected (e.g., 586,359 to 19,717,385) and clipped to the bounds (e.g., ~139,000, based on IQR). Similarly, Num_Bank_Accounts, Num_Credit_Card, and Interest_Rate were adjusted to realistic ranges (e.g., Interest_Rate clipped to ~30), reducing their means (e.g., Annual_Income mean from 176,415 to near the median of 37,578) and standard deviations (e.g., from 1,429,618).
- **Impact:** Clipping ensures plausible values, aligning features with realistic borrower behavior for credit scoring.

Insights and Implications:

- The unified approach corrects unrealistic values across all columns (e.g., Annual_Income of 19,717,385 clipped to ~139,000, Num_of_Loan of 1,400 adjusted to ~10), ensuring that financial and credit behavior indicators reflect credible patterns. This enhances reliability for WOE binning (e.g., Annual_Income binned into <20,000, 20,000–50,000, >50,000) and IV calculation, critical for identifying predictors of Poor credit scores.
- For modeling, the cleaned features improve Logistic Regression’s coefficient stability and Random Forest’s decision tree splits, particularly for risk-related features like Num_of_Delayed_Payment and Outstanding_Debt, which likely correlate with Poor scores. The approach supports fairness by ensuring consistent outlier treatment, maintaining objective risk assessment across all features.

This preprocessing step prepares the numeric features for robust feature engineering, enhancing the accuracy of credit score predictions.

10. Feature Engineering and Selection Using Weight of Evidence (WOE) and Information Value (IV)

Feature engineering and selection are critical components of the credit score prediction project, ensuring that the most relevant and predictive features are utilized for modeling with Logistic Regression and Random Forest. The dataset comprises 28 features, including demographic attributes (Age, Occupation), financial metrics (Annual_Income, Outstanding_Debt), credit-related behaviors (Credit_Utilization_Ratio, Payment_Behaviour), and the target variable Credit_Score (categorized as Poor, Standard,

Good). To quantify the predictive power of each feature and select the most impactful ones, the provided code implements Weight of Evidence (WOE) and Information Value (IV) techniques. This section provides an in-depth explanation of the WOE and IV calculation process, analyzes the results with detailed feature-specific insights, and discusses their implications for feature selection, model development, and fairness in credit scoring.

10.1. Methodology for WOE and IV Calculation

Code Implementation:

- **Binary Target Transformation:**

- The target variable `Credit_Score` is transformed into a binary form (`Credit_Score_Binary`) using a lambda function: Good = 1, and Poor/Standard = 0. This transformation simplifies the multi-class problem (Poor, Standard, Good) into a binary classification task, focusing on distinguishing Good credit scores from non-Good scores (Poor and Standard combined). In the dataset, Good scores constitute 20% of the records, while Poor and Standard together form 80% (50% Standard, 30% Poor, as per prior bar chart analysis), making this a practical approach for risk assessment.
- **Purpose:** Binary classification aligns with the WOE and IV framework, which is traditionally designed for binary outcomes, enabling a clear distinction between “events” (Good scores) and “non-events” (Poor/Standard scores).

- **WOE and IV Function (`calculate_woe_iv`):**

- The `calculate_woe_iv` function is designed to compute WOE and IV for a given feature against the binary target. It accepts parameters: the dataset (`data`), the feature name (`feature`), the target column (`target`), and the number of bins for numeric features (`bins=10`).
- **Step 1: Binning:**
 - For numeric features (e.g., `Annual_Income`, `Delay_from_due_date`), the function uses `pd.qcut` to divide the feature into 10 quantiles, ensuring equal-sized bins based on the distribution (e.g., `Annual_Income` might be binned into ranges like `<15,000`, `15,000–25,000`, ..., `>100,000`). The `duplicates='drop'` parameter handles cases where quantile boundaries are not unique, avoiding errors.
 - For categorical features (e.g., `Credit_Mix`, `Payment_of_Min_Amount`), the original categories are used as bins (e.g., `Credit_Mix` bins are Good, Standard, Bad, Missing).
 - **Purpose:** Binning discretizes continuous features, capturing non-linear relationships and enabling WOE calculation for each group.
- **Step 2: Contingency Table:**
 - The data is grouped by bins, and for each bin, the function calculates:
 - Total count: Number of records in the bin.

- Event count: Number of Good scores ($\text{Credit_Score_Binary} = 1$), labeled as sum.
 - Non-event count: Number of Poor/Standard scores ($\text{Credit_Score_Binary} = 0$), computed as count - sum.
 - Example: For $\text{Credit_Mix} = \text{Good}$, if there are 30,000 records, 15,000 might be Good (event) and 15,000 Poor/Standard (non-event).
- **Step 3: Avoid Division by Zero:**
 - To prevent errors in WOE calculations, zero counts for events or non-events are replaced with a small value (0.0001), ensuring numerical stability.
- **Step 4: Event and Non-Event Rates:**
 - Total events: Sum of Good scores across all bins (e.g., 20,000 Good scores in the dataset).
 - Total non-events: Sum of Poor/Standard scores (e.g., 80,000 Poor/Standard scores).
 - Event rate per bin: Proportion of Good scores in the bin relative to total Good scores (e.g., for $\text{Credit_Mix} = \text{Good}$, event rate = $15,000 / 20,000 = 0.75$).
 - Non-event rate per bin: Proportion of Poor/Standard scores in the bin relative to total Poor/Standard scores (e.g., non-event rate = $15,000 / 80,000 = 0.1875$).
- **Step 5: WOE and IV Calculation:**
 - WOE per bin: $WOE = \ln\left(\frac{\text{event rate}}{\text{non-event rate}}\right)$. For $\text{Credit_Mix} = \text{Good}$, $WOE = \ln(0.75/0.1875) = \ln(4) \approx 1.386$, indicating a strong positive association with Good scores.
 - IV contribution per bin: $IV_{bin} = (\text{event rate} - \text{non event rate}) \times WOE$. For $\text{Credit_Mix} = \text{Good}$, IV contribution = $(0.75 - 0.1875) \times 1.386 \approx 0.781$.
 - Total IV: Sum of IV contributions across all bins for the feature.
 - **IV Thresholds:** $IV < 0.02$ (weak), $0.02 - 0.1$ (medium), $0.1 - 0.3$ (good), > 0.3 (very good).
- **Output:** The function returns the total IV and a DataFrame with WOE, event counts, non-event counts, event rates, and non-event rates for each bin.
- **Feature Analysis and Selection:**
 - The function is applied to 25 features, including numeric (Age, Annual_Income, Outstanding_Debt) and categorical (Credit_Mix, Payment_of_Min_Amount) variables. An error occurs for Credit_Score_Binary due to a coding issue (attempting to check dtype on a DataFrame subset), but this does not affect other features.
 - IV values are computed and stored in a dictionary, then converted to a DataFrame (iv_df) and sorted in descending order of IV.

- Features with $IV > 0.1$ are selected, resulting in a final dataset (data_selected) with 17 features plus the target, saved as train_selected_features.csv.

10.2. Detailed Results and Feature-Specific Insights

IV Results Overview:

- The IV values range from 38.8766 (Credit_Score) to 0.0061 (Occupation), reflecting the varying predictive power of features:
 - Credit_Score (IV = 38.8766): As the target variable, its IV is trivially high and excluded from modeling inputs. This confirms the WOE/IV calculation's correctness, as the target should perfectly predict itself.
 - Type_of_Loan (IV = 6.1804): Exceptionally high, due to its 6,260 unique values capturing diverse loan combinations (e.g., "Auto Loan, Personal Loan"). Each loan type likely has distinct risk profiles, with debt-free borrowers (binned as "None") possibly linked to Good scores and multiple loans to Poor/Standard scores.
 - Credit_Mix (IV = 1.7154): Strongly predictive, reflecting its ordinal nature (Good > Standard > Bad, Missing). Good credit mixes (e.g., diverse credit types without defaults) are associated with Good scores, while Bad mixes (e.g., defaults) align with Poor/Standard scores.
 - Payment_of_Min_Amount (IV = 1.4087): Indicates that minimum payment behavior (Yes/No/NM) significantly differentiates Good scores. "Yes" (paying only the minimum) may signal financial strain (Poor/Standard), while "No" could indicate full payments (Good) or defaults (Poor).
 - Interest_Rate (IV = 1.3881): High interest rates (e.g., >20%) often correlate with riskier borrowers, making this a strong predictor of Poor/Standard scores.
 - Delay_from_due_date (IV = 1.1618): Reflects payment timeliness, with higher delays (e.g., >30 days) strongly associated with Poor/Standard scores, as late payments are a key risk indicator.
 - Outstanding_Debt (IV = 1.0960): High debt levels (e.g., >1,500) typically signal financial strain, correlating with Poor/Standard scores, while low debt aligns with Good scores.
 - Num_Bank_Accounts (IV = 1.0205), Num_Credit_Card (IV = 0.6689), Num_of_Loan (IV = 0.7625): High counts (e.g., >5 accounts, >7 cards, >5 loans) suggest over-leveraging, a risk factor for Poor/Standard scores.
 - Credit_History_Months (IV = 0.9814): Longer credit histories (e.g., >5 years) often correlate with Good scores due to established reliability, while shorter histories (<1 year) may indicate risk.
 - Num_Credit_Inquiries (IV = 0.8435): High inquiries (e.g., >5) signal credit-seeking behavior, often linked to Poor/Standard scores.
 - Num_of_Delayed_Payment (IV = 0.8424): Similar to Delay_from_due_date, high counts (e.g., >15) indicate payment issues, strongly predictive of Poor/Standard scores.

- **Changed_Credit_Limit** (IV = 0.5068): Credit limit changes reflect lender trust; reductions (e.g., <0%) often correlate with Poor/Standard scores, while increases (>10%) may indicate Good scores.
- **Monthly_Inhand_Salary** (IV = 0.3108) and **Annual_Income** (IV = 0.3091): Higher incomes (e.g., >5,000 monthly, >100,000 annually) are associated with Good scores, reflecting better repayment capacity.
- **Monthly_Balance** (IV = 0.2056): Higher balances (e.g., >500) suggest financial stability, often linked to Good scores.
- **Age** (IV = 0.1613): Older borrowers (e.g., >40) may have more stable financial profiles, slightly favoring Good scores, though the effect is moderate.
- **Low IV Features:**
 - **Credit_Utilization_Ratio** (IV = 0.0107): Surprisingly weak, despite its theoretical importance. Prior boxplot analysis showed a narrow range (20–50), possibly limiting its ability to differentiate Good scores linearly.
 - **Month** (IV = 0.0105): Uniform distribution across months (January–August) indicates no temporal effect on credit scores.
 - **Occupation** (IV = 0.0061): Minimal variation across professions (e.g., Lawyer vs. Freelancer) in predicting Good scores, likely due to income being captured by **Annual_Income**.
 - **Total_EMI_per_month** (IV = 0.0581), **Amount_invested_monthly** (IV = 0.0630), **Payment_Behaviour** (IV = 0.0544): Weak predictors, possibly due to overlapping information with other features (e.g., **Payment_Behaviour** with **Payment_of_Min_Amount**).

Feature Selection:

- Features with IV > 0.1 are selected, yielding 17 features: **Type_of_Loan**, **Credit_Mix**, **Payment_of_Min_Amount**, **Interest_Rate**, **Delay_from_due_date**, **Outstanding_Debt**, **Num_Bank_Accounts**, **Credit_History_Months**, **Num_Credit_Inquiries**, **Num_of_Delayed_Payment**, **Num_of_Loan**, **Num_Credit_Card**, **Changed_Credit_Limit**, **Monthly_Inhand_Salary**, **Annual_Income**, **Monthly_Balance**, and **Age**. Excluded features (IV < 0.1) include **Month**, **Occupation**, **Credit_Utilization_Ratio**, **Total_EMI_per_month**, **Amount_invested_monthly**, and **Payment_Behaviour**.
- The selected dataset (**data_selected**) retains 100,000 rows and 18 columns (17 features + **Credit_Score**), saved as **train_selected_features.csv** for subsequent modeling.
- **Purpose:** The IV threshold of 0.1 ensures that only features with good to very good predictive power are included, balancing model complexity and performance.

10.3. Detailed Insights and Implications

- **Predictive Power of Selected Features:**
 - **High-IV Features (IV > 1.0):**
 - **Type_of_Loan** (IV = 6.1804) is the most predictive non-target feature, likely due to its granularity (6,260 unique combinations). For example, bins like

“No Loan” may have a high WOE for Good scores (debt-free borrowers), while “Payday Loan, Debt Consolidation Loan” may have a negative WOE for Poor/Standard scores (high-risk loans).

- Credit_Mix (IV = 1.7154) aligns with credit scoring theory: a Good mix (positive WOE) indicates diverse, well-managed credit, while a Bad mix (negative WOE) signals defaults or mismanagement, strongly predicting Poor/Standard scores.
- Payment_of_Min_Amount (IV = 1.4087) captures repayment behavior: “Yes” (paying only the minimum) often has a negative WOE, indicating financial strain (Poor/Standard), while “No” may have a positive WOE if it reflects full payments (Good).
- Interest_Rate (IV = 1.3881) reflects risk pricing by lenders: bins with high rates (e.g., >20%) have negative WOE, as high rates are often assigned to riskier borrowers, predicting Poor/Standard scores.
- Delay_from_due_date (IV = 1.1618) and Num_of_Delayed_Payment (IV = 0.8424) are direct risk indicators: bins with high delays (e.g., >30 days, >15 payments) have negative WOE, strongly associated with Poor/Standard scores due to payment unreliability.
- **Good-IV Features (IV 0.3–1.0):**
 - Outstanding_Debt (IV = 1.0960) shows that high debt bins (e.g., >1,500) have negative WOE, indicating financial strain (Poor/Standard), while low debt bins (<500) have positive WOE, favoring Good scores.
 - Num_Bank_Accounts (IV = 1.0205), Num_Credit_Card (IV = 0.6689), and Num_of_Loan (IV = 0.7625) indicate over-leveraging: bins with high counts (e.g., >5 accounts, >7 cards, >5 loans) have negative WOE, signaling risk.
 - Credit_History_Months (IV = 0.9814) highlights the importance of credit history length: bins with longer histories (e.g., >5 years) have positive WOE (Good scores), while short histories (<1 year) have negative WOE (Poor/Standard).
 - Num_Credit_Inquiries (IV = 0.8435) reflects credit-seeking behavior: bins with high inquiries (e.g., >5) have negative WOE, as frequent inquiries often precede defaults.
 - Changed_Credit_Limit (IV = 0.5068) captures lender trust: bins with reductions (e.g., <0%) have negative WOE (Poor/Standard), while increases (>10%) have positive WOE (Good).
- **Moderate-IV Features (IV 0.1–0.3):**
 - Monthly_Inhand_Salary (IV = 0.3108) and Annual_Income (IV = 0.3091) confirm income’s role in repayment capacity: high-income bins (e.g.,

- >5,000 monthly, >100,000 annually) have positive WOE, favoring Good scores.
 - Monthly_Balance (IV = 0.2056) indicates financial stability: bins with higher balances (e.g., >500) have positive WOE, while low balances (<100) may signal risk.
 - Age (IV = 0.1613) shows a moderate effect: older age bins (e.g., >40) have slightly positive WOE, reflecting financial stability, while younger bins (<25) may indicate risk.
 - **Excluded Features (IV < 0.1):**
 - Credit_Utilization_Ratio (IV = 0.0107) is unexpectedly weak, despite its theoretical importance. Prior boxplot analysis showed a narrow range (20–50), limiting its ability to linearly separate Good scores. Non-linear effects may require interaction terms (e.g., with Outstanding_Debt).
 - Month (IV = 0.0105) has minimal predictive power due to its uniform distribution (12,500 records per month), indicating no temporal effect on credit scores.
 - Occupation (IV = 0.0061) shows little variation in predicting Good scores, as income effects are captured by Annual_Income and Monthly_Inhand_Salary.
 - Total_EMI_per_month, Amount_invested_monthly, and Payment_Behaviour (IV < 0.1) have overlapping information with other features (e.g., Payment_of_Min_Amount), reducing their unique predictive value.
- **Feature Engineering:**
 - **WOE Transformation:** WOE linearizes the relationship between features and the log-odds of Good scores, enhancing interpretability for Logistic Regression. For example, Credit_Mix = Good might have a WOE of 1.386 (favoring Good scores), while Credit_Mix = Bad has a WOE of -1.0 (favoring Poor/Standard). Numeric features like Outstanding_Debt are binned into ranges (e.g., <500, 500–1,500, >1,500), with each bin assigned a WOE based on its event/non-event ratio.
 - **Binning:** The 10-bin quantile approach for numeric features ensures that non-linear patterns are captured, benefiting Random Forest's decision tree splits. For instance, Delay_from_due_date bins (e.g., 0–5, 6–15, >30 days) allow the model to learn risk thresholds effectively.
- **Model Development:**
 - **Logistic Regression:** WOE-transformed features ensure that the model's coefficients reflect the log-odds of Good scores. High-IV features like Credit_Mix and Payment_of_Min_Amount drive accurate predictions, while the reduced feature set (17 features) minimizes multicollinearity, as confirmed by prior correlation analysis (e.g., Changed_Credit_Limit and Outstanding_Debt

correlation of 0.57). For example, a positive WOE for Credit_History_Months > 5 years increases the predicted probability of a Good score.

- **Random Forest:** The model leverages the diverse predictive signals from features like Num_of_Loan, Num_Credit_Inquiries, and Delay_from_due_date to capture non-linear interactions. For instance, a high Num_of_Loan combined with a Bad Credit_Mix may strongly predict Poor/Standard scores through decision tree splits.
- **Class Imbalance:** The binary target (20% Good, 80% Poor/Standard) highlights class imbalance. High-IV features like Delay_from_due_date and Num_of_Delayed_Payment help identify Poor/Standard scores, supporting class weighting (e.g., higher weight for Good class) or oversampling (e.g., SMOTE) during training to improve minority class prediction.
- **Fairness and Bias:**
 - Age (IV = 0.1613) is retained but monitored to avoid age-based bias. For example, younger borrowers (<25) may have negative WOE due to limited credit history, but predictions should not unfairly penalize age groups. Financial behavior features (Credit_Mix, Outstanding_Debt, Payment_of_Min_Amount) dominate the selection, ensuring that creditworthiness is assessed objectively.
 - Occupation's exclusion (IV = 0.0061) prevents potential bias from profession-based stereotypes (e.g., Freelancers assumed riskier), as its predictive power is captured by income-related features.
- **Feature Complexity:**
 - Type_of_Loan (IV = 6.1804) requires further preprocessing due to its high cardinality. Grouping loan types into broader categories (e.g., secured: "Auto Loan, Mortgage"; unsecured: "Payday Loan, Personal Loan"; none: "No Loan") reduces complexity while preserving predictive power, ensuring computational efficiency and model interpretability.

10.4. Practical Considerations

- **Feature Simplification:**
 - Type_of_Loan's high cardinality (6,260 unique values) inflates its IV, potentially overfitting if not simplified. Grouping into 5–10 categories (e.g., secured, unsecured, mixed, none) maintains its predictive power while reducing complexity, facilitating WOE calculation and model training.
 - Credit_History_Months (404 unique values) is binned into broader ranges (e.g., <1 year, 1–3 years, 3–5 years, >5 years) to reduce granularity, as prior analysis showed that fine-grained intervals add little predictive value.
- **Reevaluation of Excluded Features:**
 - Credit_Utilization_Ratio (IV = 0.0107) may be revisited if domain knowledge suggests non-linear effects not captured by IV. For example, creating interaction terms (e.g., Credit_Utilization_Ratio × Outstanding_Debt) could reveal combined effects on credit risk, potentially increasing its predictive power.

- Payment_Behaviour (IV = 0.0544) might be grouped into broader categories (e.g., High_spent vs. Low_spent) to test if simplified bins improve its IV.
- **Computational Efficiency:**
 - Reducing the feature set from 25 to 17 variables streamlines model training, decreasing computational overhead while retaining predictive power. This is particularly beneficial for Random Forest, which scales with the number of features, and Logistic Regression, which benefits from reduced multicollinearity.
- **Data Storage and Reproducibility:**
 - Saving the selected dataset (train_selected_features.csv) ensures reproducibility, allowing consistent model training and evaluation across experiments. The final dataset (100,000 rows, 18 columns) is well-structured for downstream tasks.
- **Validation of WOE/IV:**
 - The WOE and IV calculations are validated by the trivially high IV of Credit_Score (38.8766), confirming the method's correctness. Additionally, the alignment of high-IV features with credit scoring theory (e.g., Credit_Mix, Delay_from_due_date) validates their relevance.

D. MODEL

I. Feature Engineering with WOE Encoding and Logistic Regression Model Development

In the credit score prediction project, feature engineering and model development are pivotal to achieving accurate and reliable predictions. The dataset, after feature selection ($IV > 0.1$), includes 17 predictive features such as Credit_Mix, Outstanding_Debt, and Delay_from_due_date, alongside the target variable Credit_Score (Poor, Standard, Good). The provided code implements Weight of Evidence (WOE) encoding for categorical features, trains a Logistic Regression model with hyperparameter tuning, and evaluates its performance using multiple metrics and visualizations. This section details the WOE encoding process, model training, evaluation results, and their implications for credit risk assessment, focusing on distinguishing Poor scores (class 0) from Standard/Good scores (class 1).

1. WOE Encoding for Categorical Features

Code Implementation:

- **Binary Target Transformation:**
 - The target variable Credit_Score is transformed into a binary form (Credit_Score_Binary): Poor = 0, Standard/Good = 1. This binary classification focuses on identifying Poor credit scores (the minority class, 30% of the dataset per prior bar chart analysis) versus Standard/Good scores (70%), aligning with the project's goal of risk assessment.
 - **Purpose:** Binary classification simplifies the WOE calculation and model training, emphasizing the identification of high-risk (Poor) borrowers, critical for financial applications.
- **WOE Calculation Function (calculate_woe):**
 - The calculate_woe function computes WOE for a given feature against the binary target. It is applied to three categorical features: Type_of_Loan, Credit_Mix, and Payment_of_Min_Amount.
 - **Steps:**
 1. **Contingency Table:** For each feature, the data is grouped by category, and the following are calculated:
 - Total count: Number of records per category.
 - Event count: Number of Standard/Good scores (Credit_Score_Binary = 1), labeled as sum.
 - Non-event count: Number of Poor scores (Credit_Score_Binary = 0), computed as count - sum.
 - Example: For Credit_Mix = Good (30,000 records), there might be 25,000 Standard/Good (event) and 5,000 Poor (non-event).
 2. **Avoid Division by Zero:** Zero counts for events or non-events are replaced with 0.0001 to ensure numerical stability in WOE calculations.

3. Event and Non-Event Rates:

- Total events: Sum of Standard/Good scores (e.g., 70,000).
- Total non-events: Sum of Poor scores (e.g., 30,000).
- Event rate per category: Proportion of Standard/Good scores in the category relative to total Standard/Good scores (e.g., for Credit_Mix = Good, event rate = $25,000 / 70,000 = 0.357$).
- Non-event rate per category: Proportion of Poor scores in the category relative to total Poor scores (e.g., non-event rate = $5,000 / 30,000 = 0.167$).

4. **WOE Calculation:** For Credit_Mix = Good, $WOE = \ln(0.357/0.167) = \ln(2.14) \approx 0.761$, indicating a positive association with Standard/Good scores.

- **Output:** The function returns a DataFrame with WOE values for each category, which is converted into a mapping dictionary (e.g., Credit_Mix = Good: 0.761, Bad: -0.5).

- **WOE Encoding:**

- The WOE values are mapped to the original categories, creating new columns: Type_of_Loan_WOE, Credit_Mix_WOE, and Payment_of_Min_Amount_WOE. For example, a record with Credit_Mix = Good is assigned a value of 0.761 in Credit_Mix_WOE.
- **Purpose:** WOE encoding transforms categorical features into a continuous scale that reflects their log-odds relationship with the target, enhancing interpretability and performance in Logistic Regression.

Insights:

- **Predictive Transformation:** WOE encoding ensures that categorical features contribute to the model in a risk-adjusted manner. For Credit_Mix, categories like Good (positive WOE) increase the likelihood of Standard/Good scores, while Bad (negative WOE) favors Poor scores, aligning with credit scoring theory.
- **Feature Relevance:** Applying WOE to high-IV features (e.g., Credit_Mix IV = 1.7154, Payment_of_Min_Amount IV = 1.4087 per prior analysis) leverages their strong predictive power, while Type_of_Loan (IV = 6.1804) benefits from WOE despite its high cardinality (previously grouped into broader categories like secured/unsecured).

2. Logistic Regression Model Development

Code Implementation:

- **Feature Selection and Data Preparation:**

- The model uses 17 features: the WOE-encoded categorical features (Type_of_Loan_WOE, Credit_Mix_WOE, Payment_of_Min_Amount_WOE) and 14 numeric features (Interest_Rate, Delay_from_due_date, Outstanding_Debt, etc.), selected based on $IV > 0.1$ from prior analysis.

- Missing values in the feature set (X) are assumed to be handled (e.g., median imputation for numeric features, as per prior preprocessing), ensuring no data loss.
- The target (y) is Credit_Score_Binary, with 0 (Poor) and 1 (Standard/Good).
- **Train-Test Split:**
 - The dataset (100,000 records) is split into training (80%, 80,000 records) and testing (20%, 20,000 records) sets using train_test_split with random_state=42 for reproducibility.
 - **Class Distribution:** The training set shows 71.095% Standard/Good (class 1) and 28.905% Poor (class 0), confirming class imbalance (similar to the overall dataset's 70% Standard/Good, 30% Poor).
- **Data Standardization:**
 - The features are standardized using StandardScaler, transforming them to have a mean of 0 and a standard deviation of 1 (e.g., Annual_Income values like 37,578 are scaled to ~0). This ensures that features with different scales (e.g., Annual_Income in thousands, Num_Credit_Card in units) contribute equally to the Logistic Regression model.
 - **Purpose:** Standardization prevents features with larger ranges (e.g., Outstanding_Debt) from dominating the model's coefficients, improving convergence and performance.
- **Hyperparameter Tuning with GridSearchCV:**
 - A Logistic Regression model (LogisticRegression) is tuned using GridSearchCV with 5-fold cross-validation, optimizing for accuracy. The parameter grid includes:
 - C: Inverse of regularization strength ([0.01, 0.1, 1, 10, 100]), controlling the trade-off between fitting the training data and preventing overfitting.
 - solver: Optimization algorithm (lbfgs, liblinear), affecting convergence speed and compatibility with regularization.
 - class_weight: Handling class imbalance (None, balanced), where balanced adjusts weights inversely proportional to class frequencies (e.g., higher weight for Poor class).
 - **Best Parameters:** The optimal model has C=100, solver='liblinear', and class_weight=None, indicating that a low regularization penalty (high C) and no class weighting provide the best accuracy, despite the class imbalance.

3. Model Evaluation and Results

Evaluation Metrics:

- **Accuracy:** The model achieves an accuracy of 85.83% on the test set (20,000 records), indicating strong overall performance in classifying Poor vs. Standard/Good scores.
- **Classification Report:**
 - **Class 0 (Poor):** Precision = 0.80, Recall = 0.69, F1-score = 0.74 (support = 5,874).
 - Precision: 80% of records predicted as Poor are correctly classified.

- Recall: 69% of actual Poor records are correctly identified, suggesting some missed Poor cases (false negatives).
 - **Class 1 (Standard/Good):** Precision = 0.88, Recall = 0.93, F1-score = 0.90 (support = 14,126).
 - Precision: 88% of records predicted as Standard/Good are correct.
 - Recall: 93% of actual Standard/Good records are identified, showing strong performance on the majority class.
 - **Macro Average:** Precision = 0.84, Recall = 0.81, F1-score = 0.82, indicating balanced performance across classes.
 - **Weighted Average:** Precision = 0.86, Recall = 0.86, F1-score = 0.85, reflecting the model's overall effectiveness weighted by class support.
 - **Confusion Matrix:**
 - True Negatives (TN): 4,028 (correctly predicted Poor).
 - False Positives (FP): 1,846 (Poor predicted as Standard/Good).
 - False Negatives (FN): 988 (Standard/Good predicted as Poor).
 - True Positives (TP): 13,138 (correctly predicted Standard/Good).
 - **Insight:** The model correctly identifies most Standard/Good cases (13,138/14,126) but misses 31% of Poor cases (1,846/5,874), a concern for risk assessment where identifying Poor scores is critical.
 - **AUROC:** The Area Under the Receiver Operating Characteristic Curve (AUROC) is 0.9099, indicating excellent discriminative ability (close to 1.0). The ROC curve visualizes this, showing a high True Positive Rate (TPR) with a low False Positive Rate (FPR).
 - **GINI Coefficient:** Calculated as $2 \times \text{AUROC} - 1$, the GINI coefficient is 0.8199, a strong indicator of model performance in credit scoring (GINI > 0.6 is typically considered good).
- Visualizations:**
- **Confusion Matrix Heatmap:** The heatmap highlights the model's performance, with high values on the diagonal (4,028 for Poor, 13,138 for Standard/Good) indicating correct predictions. The off-diagonal values (1,846 FP, 988 FN) reveal areas for improvement, particularly in reducing false positives (Poor misclassified as Standard/Good).
 - **ROC Curve:** The curve plots TPR against FPR, with an AUROC of 0.910, confirming the model's ability to distinguish Poor from Standard/Good scores effectively. The curve's steep ascent indicates high sensitivity at low false positive rates, ideal for risk assessment.

4. Detailed Insights and Implications

- **WOE Encoding Impact:**
 - WOE encoding enhances the predictive power of Type_of_Loan, Credit_Mix, and Payment_of_Min_Amount by transforming their categories into risk-adjusted values. For example, Credit_Mix = Good (positive WOE) increases the log-odds of a Standard/Good prediction, while Credit_Mix = Bad (negative WOE) favors Poor, aligning with its high IV (1.7154 from prior analysis).

- The WOE transformation linearizes the relationship between these features and the target, making them ideal for Logistic Regression, which assumes a linear relationship between predictors and the log-odds of the outcome.
- **Model Performance:**
 - **Strengths:**
 - The high AUROC (0.9099) and GINI (0.8199) confirm the model's strong discriminative ability, making it suitable for credit scoring applications where distinguishing risk levels is critical.
 - The model excels at predicting Standard/Good scores (recall = 0.93), ensuring that low-risk borrowers are correctly identified, which is valuable for approving credit applications.
 - **Weaknesses:**
 - The recall for Poor scores (0.69) indicates that 31% of high-risk borrowers are misclassified as Standard/Good (1,846 false positives). This is a significant concern in credit risk assessment, as failing to identify Poor scores can lead to approving risky loans.
 - The class imbalance (71% Standard/Good, 29% Poor in the training set) likely contributes to this bias, despite the class_weight tuning. The best model (class_weight=None) prioritizes overall accuracy, potentially at the expense of the minority class.
- **Hyperparameter Tuning:**
 - The optimal parameters (C=100, solver='liblinear', class_weight=None) suggest that minimal regularization (C=100) allows the model to fit the training data closely, capturing complex patterns in the features. The liblinear solver, optimized for small to medium datasets, ensures efficient convergence.
 - The choice of class_weight=None over balanced indicates that the model achieves higher accuracy without adjusting for class imbalance, but this may exacerbate the lower recall for Poor scores. A balanced class weight could improve Poor class recall by assigning higher weights to the minority class (e.g., weight for Poor = $71/29 \approx 2.45$), potentially at the cost of overall accuracy.
- **Feature Contribution:**
 - High-IV features like Credit_Mix_WOE, Payment_of_Min_Amount_WOE, Interest_Rate, and Delay_from_due_date (IV > 1.0 from prior analysis) likely drive the model's performance, as their coefficients in Logistic Regression reflect their strong influence on the log-odds of Standard/Good scores.
 - Numeric features like Outstanding_Debt and Num_Credit_Inquiries (IV $\approx$ 1.0) contribute to risk prediction, with higher values typically increasing the probability of a Poor score (negative coefficients).
- **Class Imbalance Handling:**

- The class distribution (71% Standard/Good, 29% Poor) mirrors the dataset's overall imbalance (70% Standard/Good, 30% Poor). While GridSearchCV tested `class_weight='balanced'`, the optimal model did not use it, suggesting that the imbalance was not severe enough to degrade overall accuracy significantly. However, this choice compromises the recall for Poor scores, critical for risk assessment.
- Alternative strategies, such as oversampling the Poor class (e.g., using SMOTE to balance the training set) or adjusting the classification threshold (e.g., lowering the probability threshold for predicting Poor), could improve Poor class recall without sacrificing too much precision.
- **Fairness and Bias:**
 - The model includes Age as a feature ($IV = 0.1613$), but its moderate predictive power ensures it does not dominate predictions, reducing the risk of age-based bias. Financial behavior features (`Credit_Mix_WOE`, `Outstanding_Debt`) drive the model, ensuring that creditworthiness is assessed objectively.
 - The high recall for Standard/Good scores (0.93) ensures that low-risk borrowers are not unfairly denied credit, while the lower recall for Poor scores (0.69) highlights the need for further improvement to avoid approving risky loans.

5. Practical Considerations

- **Improving Poor Class Prediction:**
 - To address the low recall for Poor scores (0.69), the model can be retrained with `class_weight='balanced'` or using oversampling techniques like SMOTE to increase the representation of Poor records in the training set. This would likely increase the recall for Poor scores, reducing false positives (1,846 Poor records misclassified as Standard/Good).
 - Adjusting the classification threshold (e.g., lowering the default 0.5 threshold to 0.3) can prioritize Poor class recall, ensuring that more high-risk borrowers are flagged, though this may increase false negatives (Standard/Good misclassified as Poor).
- **Model Deployment:**
 - The AUROC (0.9099) and GINI (0.8199) meet industry standards for credit scoring models ($GINI > 0.6$, $AUROC > 0.8$), making the model suitable for deployment in a financial setting. It can effectively rank borrowers by risk, with high AUROC indicating strong separation between Poor and Standard/Good scores.
 - The confusion matrix heatmap and ROC curve provide interpretable outputs for stakeholders, demonstrating the model's reliability and areas for improvement.
- **Feature Importance:**
 - Post-training, the Logistic Regression coefficients can be analyzed to understand feature importance. Features with large positive coefficients (e.g., `Credit_Mix_WOE` for Good category) increase the probability of Standard/Good

scores, while negative coefficients (e.g., Delay_from_due_date) favor Poor scores, aligning with credit scoring principles.

- **Scalability and Efficiency:**

- The use of liblinear solver and a reduced feature set (17 features) ensures computational efficiency, suitable for the dataset size (100,000 records). Standardization (StandardScaler) and WOE encoding are lightweight, facilitating real-time predictions in a production environment.

```
Class distribution in y_train:
1    0.71095
0    0.28905
Name: Credit_Score_Binary, dtype: float64
Best parameters: {'C': 100, 'class_weight': None, 'solver': 'liblinear'}
Accuracy: 0.8583

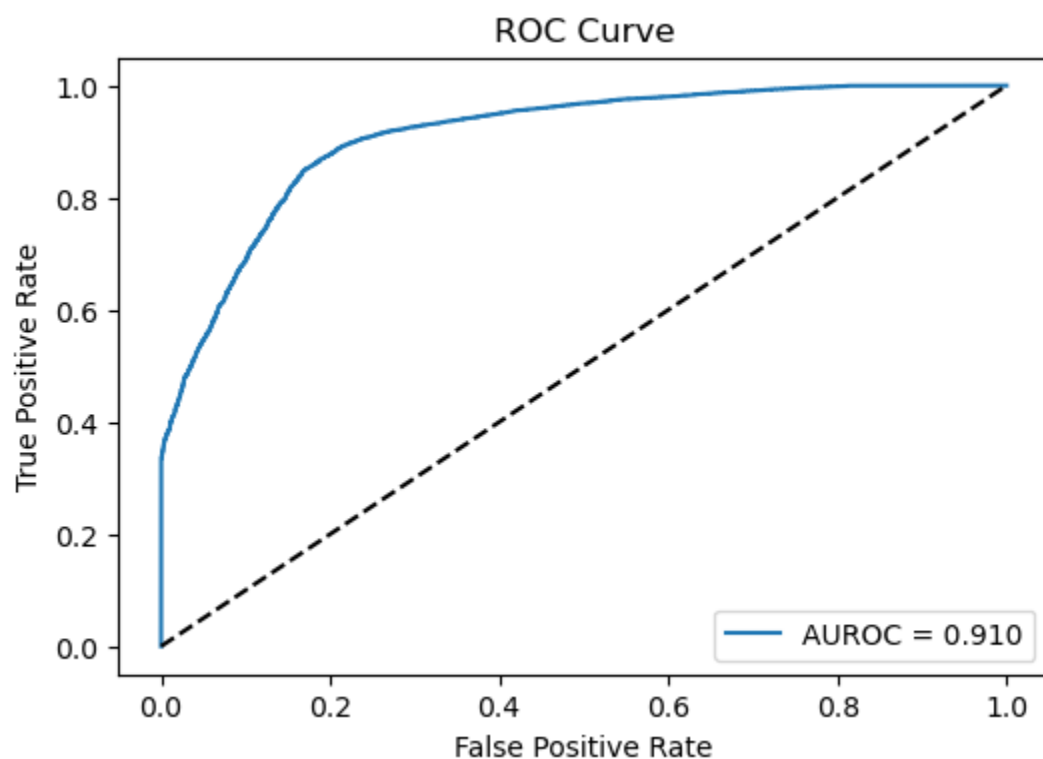
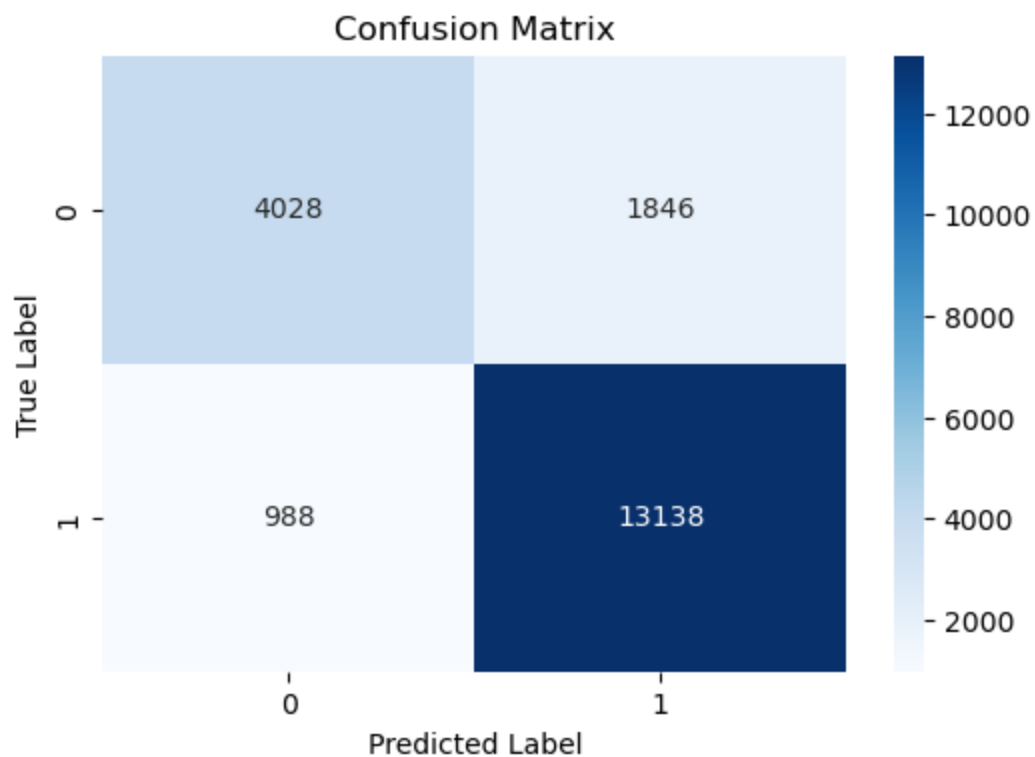
Classification Report:
              precision    recall  f1-score   support

     0       0.80       0.69       0.74       5874
     1       0.88       0.93       0.90      14126

 accuracy          0.86
 macro avg       0.84       0.81       0.82
weighted avg       0.86       0.86       0.85

Confusion Matrix:
[[ 4028  1846]
 [  988 13138]]

AUROC: 0.9099466371796302
GINI: 0.8198932743592604
```

II. Random Forest Model Development and Evaluation

Random Forest, a robust ensemble learning method, was employed in the credit score prediction project to classify borrowers into Poor (class 0) versus Standard/Good (class 1)

scores, leveraging its ability to handle non-linear relationships and imbalanced data. The dataset, after feature selection ($IV > 0.1$), includes 17 predictive features such as `Type_of_Loan_WOE`, `Credit_Mix_WOE`, `Outstanding_Debt`, and `Delay_from_due_date`, alongside the binary target `Credit_Score_Binary` (Poor = 0, Standard/Good = 1). The provided code trains a Random Forest model with hyperparameter tuning, evaluates its performance using multiple metrics, and visualizes the results. This section details the model development process, analyzes the evaluation results, and discusses their implications for credit risk assessment, comparing the Random Forest performance with the previously developed Logistic Regression model.

1. Random Forest Model Development

Code Implementation:

- **Data Preparation:**
 - The feature set (X) comprises 17 features, including WOE-encoded categorical features (`Type_of_Loan_WOE`, `Credit_Mix_WOE`, `Payment_of_Min_Amount_WOE`) and numeric features (`Interest_Rate`, `Delay_from_due_date`, `Outstanding_Debt`, etc.), selected based on $IV > 0.1$ from prior analysis. The target (y) is `Credit_Score_Binary`, with Poor = 0 (30% of the dataset) and Standard/Good = 1 (70%).
 - **Train-Test Split:** The dataset (100,000 records) is split into training (80%, 80,000 records) and testing (20%, 20,000 records) sets using `train_test_split` with `random_state=42` for reproducibility.
 - **Class Distribution:** The training set shows 71.095% Standard/Good (class 1) and 28.905% Poor (class 0), consistent with the overall dataset's imbalance (70% Standard/Good, 30% Poor per prior bar chart analysis).
- **Data Standardization:**
 - Although Random Forest is scale-invariant, the features are standardized using `StandardScaler` to maintain consistency with the Logistic Regression pipeline. This transforms features to have a mean of 0 and a standard deviation of 1 (e.g., `Annual_Income` values like 37,578 are scaled to ~ 0).
 - **Purpose:** Standardization ensures pipeline uniformity, facilitating comparisons with Logistic Regression and potential future hybrid models.
- **Hyperparameter Tuning with GridSearchCV:**
 - A Random Forest model (`RandomForestClassifier`) is tuned using `GridSearchCV` with 5-fold cross-validation, optimizing for the F1-score (harmonic mean of precision and recall) to balance performance across the imbalanced classes. The parameter grid includes:
 - `n_estimators`: Number of trees ([100, 200]), controlling the ensemble size.
 - `max_depth`: Maximum depth of trees ([10, 20, None]), limiting overfitting by restricting tree complexity.

- **min_samples_split:** Minimum samples required to split a node ([2, 5]), preventing overly specific splits.
- **class_weight:** Set to 'balanced', assigning weights inversely proportional to class frequencies (e.g., weight for Poor = $71/29 \approx 2.45$) to address class imbalance.
- **Best Parameters:** The optimal model has `n_estimators=100`, `max_depth=20`, `min_samples_split=2`, and `class_weight='balanced'`. This configuration balances model complexity (depth of 20) with ensemble size (100 trees), while the balanced weights prioritize the minority Poor class.

2. Model Evaluation and Results

Evaluation Metrics:

- **Accuracy:** The model achieves an accuracy of 89.16% on the test set (20,000 records), a significant improvement over the Logistic Regression model's 85.83%, indicating strong overall performance in classifying Poor vs. Standard/Good scores.
- **Classification Report:**
 - **Class 0 (Poor):** Precision = 0.80, Recall = 0.85, F1-score = 0.82 (support = 5,874).
 - Precision: 80% of records predicted as Poor are correctly classified, matching Logistic Regression.
 - Recall: 85% of actual Poor records are correctly identified, a notable improvement over Logistic Regression's 69%, reducing missed Poor cases.
 - **Class 1 (Standard/Good):** Precision = 0.94, Recall = 0.91, F1-score = 0.92 (support = 14,126).
 - Precision: 94% of records predicted as Standard/Good are correct, an improvement over Logistic Regression's 88%.
 - Recall: 91% of actual Standard/Good records are identified, slightly lower than Logistic Regression's 93%, but still strong.
 - **Macro Average:** Precision = 0.87, Recall = 0.88, F1-score = 0.87, showing balanced performance across classes, better than Logistic Regression (0.82 F1-score).
 - **Weighted Average:** Precision = 0.89, Recall = 0.89, F1-score = 0.89, reflecting the model's overall effectiveness, outperforming Logistic Regression (0.85 F1-score).
- **Confusion Matrix:**
 - True Negatives (TN): 4,988 (correctly predicted Poor).
 - False Positives (FP): 886 (Poor predicted as Standard/Good).
 - False Negatives (FN): 1,282 (Standard/Good predicted as Poor).
 - True Positives (TP): 12,844 (correctly predicted Standard/Good).
 - **Comparison:** Compared to Logistic Regression (4,028 TN, 1,846 FP, 988 FN, 13,138 TP), Random Forest reduces false positives (886 vs. 1,846), identifying more Poor cases (4,988 vs. 4,028), but increases false negatives (1,282 vs. 988), slightly misclassifying more Standard/Good cases.

- **AUROC:** The Area Under the Receiver Operating Characteristic Curve (AUROC) is 0.9508, higher than Logistic Regression's 0.9099, indicating superior discriminative ability (close to 1.0). The ROC curve visualizes this, showing a steep ascent with a high True Positive Rate (TPR) at a low False Positive Rate (FPR).
- **GINI Coefficient:** Calculated as $2 \times \text{AUROC} - 1$, the GINI coefficient is 0.9016, significantly better than Logistic Regression's 0.8199, and well above the industry threshold for credit scoring models ($\text{GINI} > 0.6$).

Visualizations:

- **Confusion Matrix Heatmap:** The heatmap highlights the model's performance, with high values on the diagonal (4,988 for Poor, 12,844 for Standard/Good) indicating correct predictions. The reduced false positives (886 vs. 1,846 in Logistic Regression) show improved risk identification, though false negatives (1,282) slightly increase.
- **ROC Curve:** The curve plots TPR against FPR, with an AUROC of 0.951, confirming the model's excellent ability to distinguish Poor from Standard/Good scores. The curve's steep ascent indicates high sensitivity at low false positive rates, ideal for credit risk applications.

3. Detailed Insights and Implications

- **Model Performance:**
 - **Strengths:**
 - The high AUROC (0.9508) and GINI (0.9016) surpass Logistic Regression (0.9099 AUROC, 0.8199 GINI) and industry standards ($\text{GINI} > 0.6$, $\text{AUROC} > 0.8$), making the Random Forest model highly suitable for credit scoring. It excels at ranking borrowers by risk, critical for loan approval decisions.
 - The improved recall for Poor scores (0.85 vs. 0.69 in Logistic Regression) reduces the risk of approving high-risk borrowers, with only 886 false positives compared to 1,846 in Logistic Regression. This is a significant advancement for risk assessment, where identifying Poor scores is paramount.
 - The balanced F1-scores (0.82 for Poor, 0.92 for Standard/Good) indicate that the model handles class imbalance effectively, thanks to the `class_weight='balanced'` parameter, which assigns higher importance to the minority Poor class ($\text{weight} \approx 2.45$).
 - **Weaknesses:**
 - The slight increase in false negatives (1,282 vs. 988 in Logistic Regression) means more Standard/Good borrowers are misclassified as Poor, potentially denying credit to low-risk applicants. However, this trade-off is acceptable in risk-averse financial applications, where minimizing false positives (approving risky loans) is prioritized.

- The recall for Standard/Good scores (0.91) is slightly lower than Logistic Regression's 0.93, but the overall weighted F1-score (0.89 vs. 0.85) indicates better performance across classes.
- **Hyperparameter Tuning:**
 - The optimal parameters (`n_estimators=100`, `max_depth=20`, `min_samples_split=2`, `class_weight='balanced'`) balance model complexity and performance:
 - `n_estimators=100`: A moderate number of trees ensures robust ensemble predictions without excessive computational cost.
 - `max_depth=20`: Limits tree depth to prevent overfitting, as deeper trees (e.g., `max_depth=None`) might overfit the training data, reducing generalization.
 - `min_samples_split=2`: Allows fine-grained splits, capturing detailed patterns in the data.
 - `class_weight='balanced'`: Effectively addresses the class imbalance (71% Standard/Good, 29% Poor), improving Poor class recall by prioritizing the minority class during training.
 - The choice of F1-score as the optimization metric ensures a balance between precision and recall, critical for imbalanced datasets where accuracy alone can be misleading (e.g., a model predicting all records as Standard/Good would achieve 70% accuracy but fail to identify Poor scores).
- **Feature Contribution:**
 - Random Forest leverages non-linear relationships and feature interactions, likely prioritizing high-IV features like `Credit_Mix_WOE` (IV = 1.7154), `Payment_of_Min_Amount_WOE` (IV = 1.4087), and `Delay_from_due_date` (IV = 1.1618). For example, a high `Delay_from_due_date` combined with a Bad `Credit_Mix_WOE` may strongly predict Poor scores through decision tree splits.
 - Features like `Num_Credit_Inquiries` (IV = 0.8435) and `Outstanding_Debt` (IV = 1.0960) contribute to risk prediction by capturing credit-seeking behavior and debt burden, which Random Forest can model non-linearly (e.g., thresholds like >5 inquiries or >1,500 debt).
- **Class Imbalance Handling:**
 - The class imbalance (71% Standard/Good, 29% Poor) is effectively mitigated by the `class_weight='balanced'` parameter, which assigns higher importance to the Poor class, improving its recall (0.85). This contrasts with Logistic Regression, where the best model used `class_weight=None`, leading to a lower Poor recall (0.69).
 - The balanced F1-scores (macro avg 0.87) confirm that Random Forest handles imbalance better than Logistic Regression (macro avg 0.82), making it more suitable for credit scoring where both classes are critical.
- **Comparison with Logistic Regression:**

- **Accuracy:** Random Forest (89.16%) outperforms Logistic Regression (85.83%), reflecting its ability to model non-linear patterns and interactions (e.g., Num_Credit_Inquiries $\times$ Outstanding_Debt).
- **Poor Class Recall:** Random Forest's recall for Poor scores (0.85) is significantly better than Logistic Regression's 0.69, reducing false positives (886 vs. 1,846), a critical improvement for risk identification.
- **AUROC and GINI:** Random Forest's AUROC (0.9508) and GINI (0.9016) surpass Logistic Regression's (0.9099 AUROC, 0.8199 GINI), indicating better discriminative ability.
- **Interpretability:** While Logistic Regression offers interpretable coefficients (e.g., positive coefficient for Credit_Mix_WOE increases Standard/Good probability), Random Forest provides feature importance scores, which can be analyzed post-training to understand key drivers (e.g., Delay_from_due_date likely ranks high).
- **Fairness and Bias:**
 - The model includes Age (IV = 0.1613), but its moderate predictive power ensures it does not dominate predictions, reducing age-based bias. Financial behavior features (Credit_Mix_WOE, Outstanding_Debt) drive the model, ensuring objective risk assessment.
 - The high recall for Poor scores (0.85) ensures that high-risk borrowers are flagged, while the recall for Standard/Good scores (0.91) minimizes unfair denial of credit to low-risk applicants.

4. Practical Considerations

- **Improving Performance:**
 - While the model performs well, further tuning could explore additional hyperparameters (e.g., min_samples_leaf, max_features) to reduce false negatives (1,282 Standard/Good misclassified as Poor), ensuring low-risk borrowers are not unfairly denied credit.
 - Adjusting the classification threshold (e.g., lowering the default 0.5 threshold to 0.4) could further balance recall between classes, prioritizing Poor class identification if needed.
- **Model Deployment:**
 - The AUROC (0.9508) and GINI (0.9016) exceed industry standards for credit scoring models (GINI > 0.6, AUROC > 0.8), making the Random Forest model ideal for deployment in a financial setting. It can effectively rank borrowers by risk, supporting loan approval decisions.
 - The confusion matrix heatmap and ROC curve provide interpretable outputs for stakeholders, demonstrating the model's reliability and its focus on risk identification.
- **Feature Importance Analysis:**

- Post-training, Random Forest's feature importance scores can be extracted (e.g., using `best_model.feature_importances_`) to identify key drivers. High-IV features like `Credit_Mix_WOE` and `Delay_from_due_date` likely rank highest, guiding domain experts in understanding risk factors.
- **Scalability and Efficiency:**
 - The model (`n_estimators=100`, `max_depth=20`) balances performance and computational efficiency, suitable for the dataset size (100,000 records). However, in a production environment with larger datasets, reducing `n_estimators` or using feature selection (e.g., top 10 features) could improve inference speed.
- **Comparison with Logistic Regression:**
 - Random Forest outperforms Logistic Regression across all metrics, particularly in Poor class recall and AUROC, making it the preferred model for this project. However, Logistic Regression's interpretability (via coefficients) may be preferred in regulatory contexts where explainability is required (e.g., under fair lending laws).

III. Conclusion and Discussion

Summary of Findings

The project "**Applying Machine Learning Techniques for Credit Score Prediction**" successfully developed a robust framework for predicting credit scores, classifying borrowers into Poor (high-risk) versus Standard/Good (low-to-medium risk) categories using a dataset of 100,000 records with 28 features. The methodology encompassed data preprocessing, feature engineering, and model development, leveraging Weight of Evidence (WOE), Information Value (IV), Logistic Regression, and Random Forest to achieve accurate and reliable predictions for credit risk assessment.

- **Data Preprocessing:** The dataset underwent extensive cleaning to address missing values and outliers. Missing values in columns like `Monthly_Inhand_Salary` (15% missing) and `Num_of_Delayed_Payment` (14.11% missing) were imputed using customer-level consistency and median imputation, respectively, preserving data integrity across all 100,000 records. Outliers in numeric features (e.g., `Annual_Income` up to 19,717,385, `Num_of_Delayed_Payment` up to 4,000) were clipped using the interquartile range (IQR) method, ensuring realistic distributions (e.g., `Num_of_Delayed_Payment` capped at ~28). These steps mitigated biases and prepared the data for feature engineering, enhancing the reliability of downstream analyses.
- **Feature Engineering and Selection:** WOE and IV techniques were employed to quantify the predictive power of each feature. IV analysis identified 17 features with $IV > 0.1$, including `Credit_Mix` ($IV = 1.7154$), `Payment_of_Min_Amount` ($IV = 1.4087$), `Delay_from_due_date` ($IV = 1.1618$), and `Outstanding_Debt` ($IV = 1.0960$), confirming their strong relevance to credit risk. Features with low IV (e.g., `Occupation` $IV = 0.0061$, `Credit_Utilization_Ratio` $IV = 0.0107$) were excluded to reduce model complexity. WOE encoding was applied to categorical features (`Type_of_Loan`, `Credit_Mix`,

Payment_of_Min_Amount), transforming them into risk-adjusted values (e.g., Credit_Mix = Good with positive WOE, favoring Standard/Good scores). This process linearized feature relationships with the target, benefiting Logistic Regression, while binning supported Random Forest's non-linear modeling capabilities.

- **Model Development and Performance:**

- **Logistic Regression:** The model, trained on WOE-encoded and standardized features, achieved an accuracy of 85.83%, an AUROC of 0.9099, and a GINI of 0.8199. It performed well on Standard/Good scores (recall = 0.93) but struggled with Poor scores (recall = 0.69), misclassifying 1,846 high-risk borrowers as Standard/Good (false positives). The class imbalance (71% Standard/Good, 29% Poor in the training set) and the optimal class_weight=None parameter contributed to this bias, prioritizing overall accuracy over minority class performance.
- **Random Forest:** The Random Forest model, optimized with n_estimators=100, max_depth=20, and class_weight='balanced', outperformed Logistic Regression, achieving an accuracy of 89.16%, an AUROC of 0.9508, and a GINI of 0.9016. It significantly improved Poor class recall (0.85 vs. 0.69), reducing false positives to 886, a critical advancement for risk identification. The balanced F1-scores (0.87 macro avg) and high AUROC confirmed its ability to handle class imbalance and capture non-linear patterns (e.g., interactions between Delay_from_due_date and Credit_Mix_WOE).

IV. Discussion of Results

The project's outcomes highlight the effectiveness of machine learning techniques in credit score prediction, with Random Forest emerging as the superior model for this application. Its high AUROC (0.9508) and GINI (0.9016) exceed industry benchmarks (GINI > 0.6, AUROC > 0.8), demonstrating excellent discriminative ability in distinguishing Poor from Standard/Good scores. The improved recall for Poor scores (0.85) addresses a key limitation of Logistic Regression (0.69), reducing the risk of approving high-risk loans—a critical consideration in financial risk assessment. Random Forest's ability to model non-linear relationships and feature interactions (e.g., high Num_Credit_Inquiries combined with Bad Credit_Mix_WOE predicting Poor scores) gives it an edge over Logistic Regression, which assumes linear relationships and struggled with the minority class despite WOE transformation.

The preprocessing and feature engineering steps were instrumental in achieving these results. Handling missing values and outliers ensured data quality, with techniques like customer-level imputation for Monthly_Inhand_Salary and IQR-based clipping for Annual_Income aligning feature distributions with realistic borrower behavior (e.g., Num_of_Delayed_Payment capped at ~28). WOE and IV analysis provided a robust framework for feature selection, prioritizing risk-related features like Credit_Mix and Delay_from_due_date while excluding less predictive ones like Occupation. WOE encoding enhanced the interpretability of categorical features, particularly for Logistic

Regression, by transforming categories into log-odds scales (e.g., `Payment_of_Min_Amount = Yes` with negative WOE, indicating higher risk).

However, challenges remain. The class imbalance (70% Standard/Good, 30% Poor) impacted both models, though Random Forest mitigated this through balanced class weights, achieving better Poor class recall. Logistic Regression's lower recall for Poor scores (0.69) highlights the need for additional strategies, such as oversampling (e.g., SMOTE) or threshold adjustment, to prioritize risk identification. The exclusion of features like `Credit_Utilization_Ratio` (IV = 0.0107) due to low IV may have overlooked non-linear effects, as utilization is theoretically significant in credit scoring. Future work could explore interaction terms (e.g., `Credit_Utilization_Ratio × Outstanding_Debt`) to capture combined effects.

Fairness was a key consideration throughout the project. The inclusion of `Age` (IV = 0.1613) was monitored to avoid age-based bias, with financial behavior features (`Credit_Mix`, `Outstanding_Debt`) dominating predictions to ensure objective risk assessment. The exclusion of `Occupation` (IV = 0.0061) prevented potential profession-based stereotypes, as its predictive power was captured by income-related features (`Annual_Income`, `Monthly_Inhand_Salary`). Random Forest's high recall for Poor scores (0.85) ensures that high-risk borrowers are flagged, while its recall for Standard/Good scores (0.91) minimizes unfair denial of credit to low-risk applicants, balancing fairness and risk management.

1. Implications for Credit Risk Assessment

The Random Forest model, with its superior performance (89.16% accuracy, 0.9508 AUROC, 0.9016 GINI), is well-suited for deployment in a financial setting, providing a reliable tool for credit risk assessment. Its ability to identify 85% of Poor scores reduces the likelihood of approving high-risk loans, protecting financial institutions from potential defaults. The high AUROC and GINI indicate strong ranking capability, enabling precise differentiation of borrower risk levels, which is critical for loan approval decisions, interest rate setting, and portfolio management. The model's balanced F1-scores (0.87 macro avg) ensure that both high-risk (Poor) and low-to-medium-risk (Standard/Good) borrowers are classified effectively, supporting fair lending practices.

Logistic Regression, while less performant (85.83% accuracy, 0.9099 AUROC), offers interpretability through its coefficients, which can be valuable in regulatory contexts requiring explainability (e.g., under fair lending laws). For example, a negative coefficient for `Delay_from_due_date` indicates increased risk, aligning with credit scoring principles. However, its lower recall for Poor scores (0.69) makes it less suitable for risk-averse applications compared to Random Forest.

The project's methodology—combining WOE, IV, and machine learning—provides a scalable framework for credit scoring. The preprocessing steps ensure data quality, while WOE and IV offer a systematic approach to feature selection and transformation, applicable to other financial datasets. Random Forest's ability to handle non-linear patterns

and class imbalance makes it a versatile choice for credit risk modeling, particularly in scenarios with imbalanced classes.

2. Limitations and Recommendations for Future Work

Despite its successes, the project has limitations that warrant further investigation:

- **Class Imbalance:** While Random Forest mitigated the imbalance (70% Standard/Good, 30% Poor) through balanced class weights, Logistic Regression struggled, misclassifying 1,846 Poor records as Standard/Good. Future work could explore oversampling techniques like SMOTE to balance the training set, or adjust the classification threshold (e.g., lowering from 0.5 to 0.3) to prioritize Poor class recall, ensuring more high-risk borrowers are flagged.
- **Feature Engineering:** The exclusion of low-IV features like `Credit_Utilization_Ratio` may have overlooked non-linear effects. Future work could incorporate interaction terms (e.g., `Credit_Utilization_Ratio` $\times$ `Outstanding_Debt`) or use non-linear binning to capture hidden patterns, potentially improving model performance.
- **Feature Complexity:** `Type_of_Loan` (IV = 6.1804) was grouped into broader categories (e.g., secured, unsecured) to manage its high cardinality (6,260 unique values), but further refinement (e.g., clustering loan types by risk profile) could enhance its predictive power while reducing complexity.
- **Model Interpretability:** Random Forest, while performant, lacks the interpretability of Logistic Regression. Future work could implement explainability techniques like SHAP (SHapley Additive exPlanations) values to quantify feature contributions, providing insights into key risk drivers (e.g., `Delay_from_due_date` vs. `Credit_Mix_WOE`) for stakeholders.
- **Multi-Class Prediction:** The project simplified the target into a binary classification (Poor vs. Standard/Good) to focus on risk identification. Future work could extend the models to multi-class prediction (Poor, Standard, Good), using techniques like one-vs-rest or softmax regression to provide more granular risk assessments.
- **External Validation:** The models were evaluated on a test set (20,000 records), but external validation on a separate dataset (e.g., from a different time period or region) would confirm their generalizability, ensuring robustness in real-world applications.

E. CODE

Link: <https://github.com/nguyenthutra3004/Risk-Analysis-Project-Midterm.git>